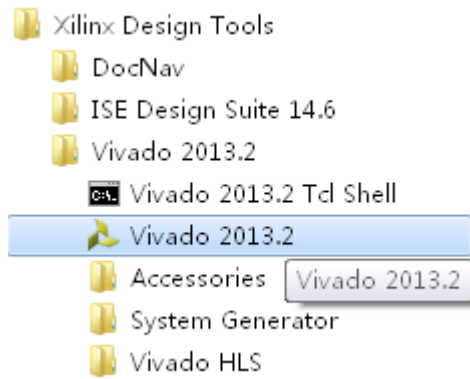


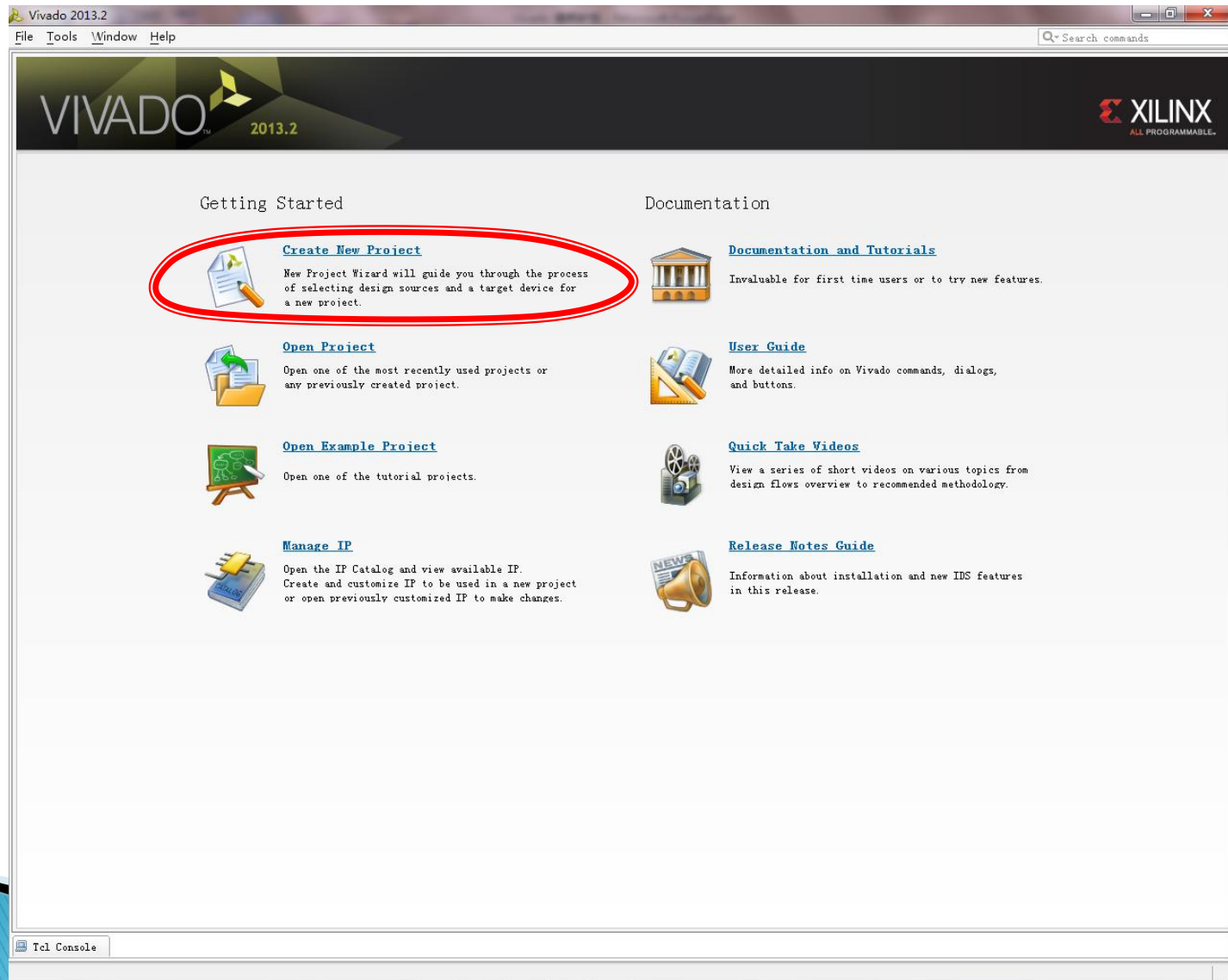
Vivado 简明教程

AVNET 南京办
蔡键龙

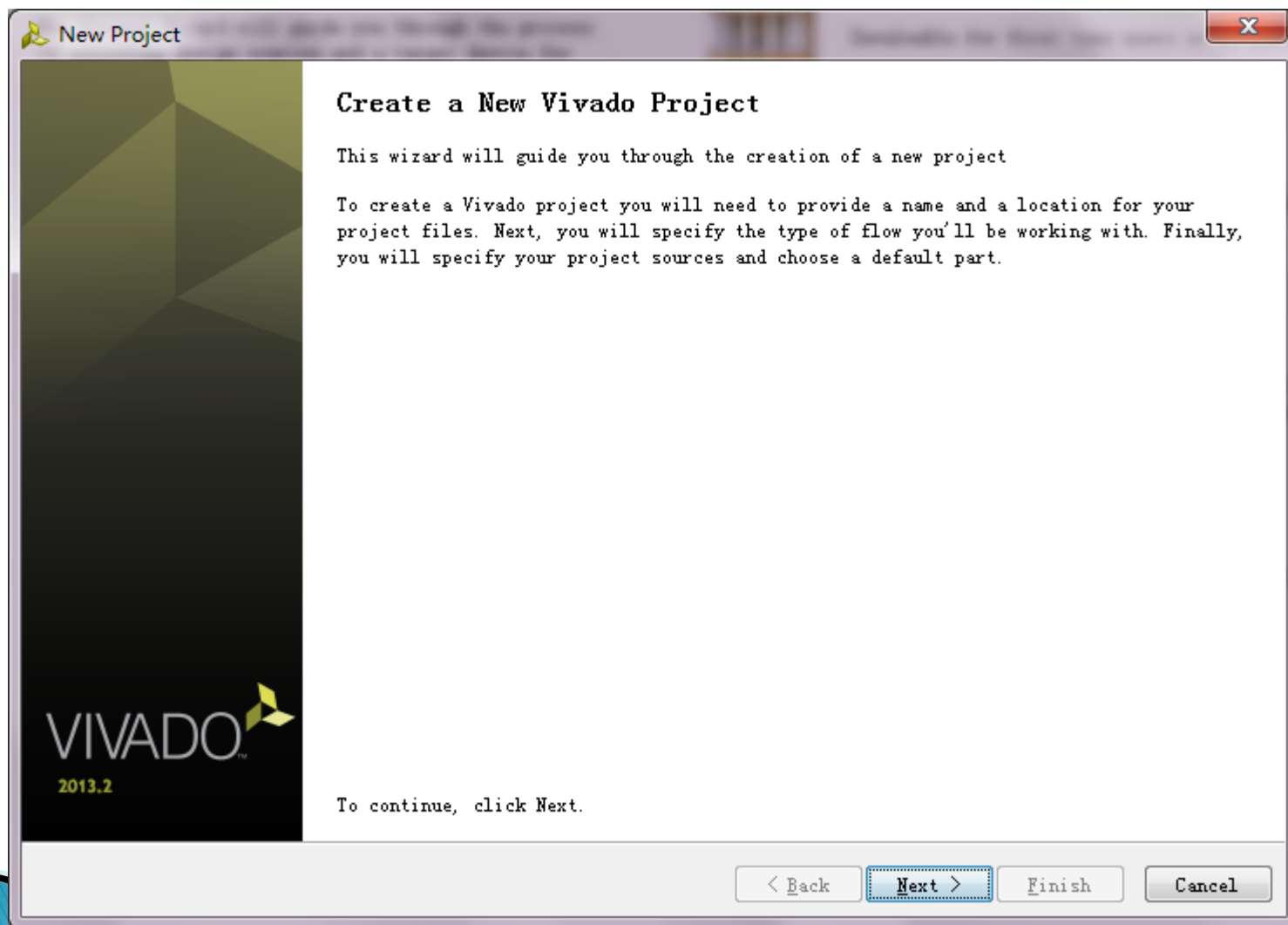
启动Vivado



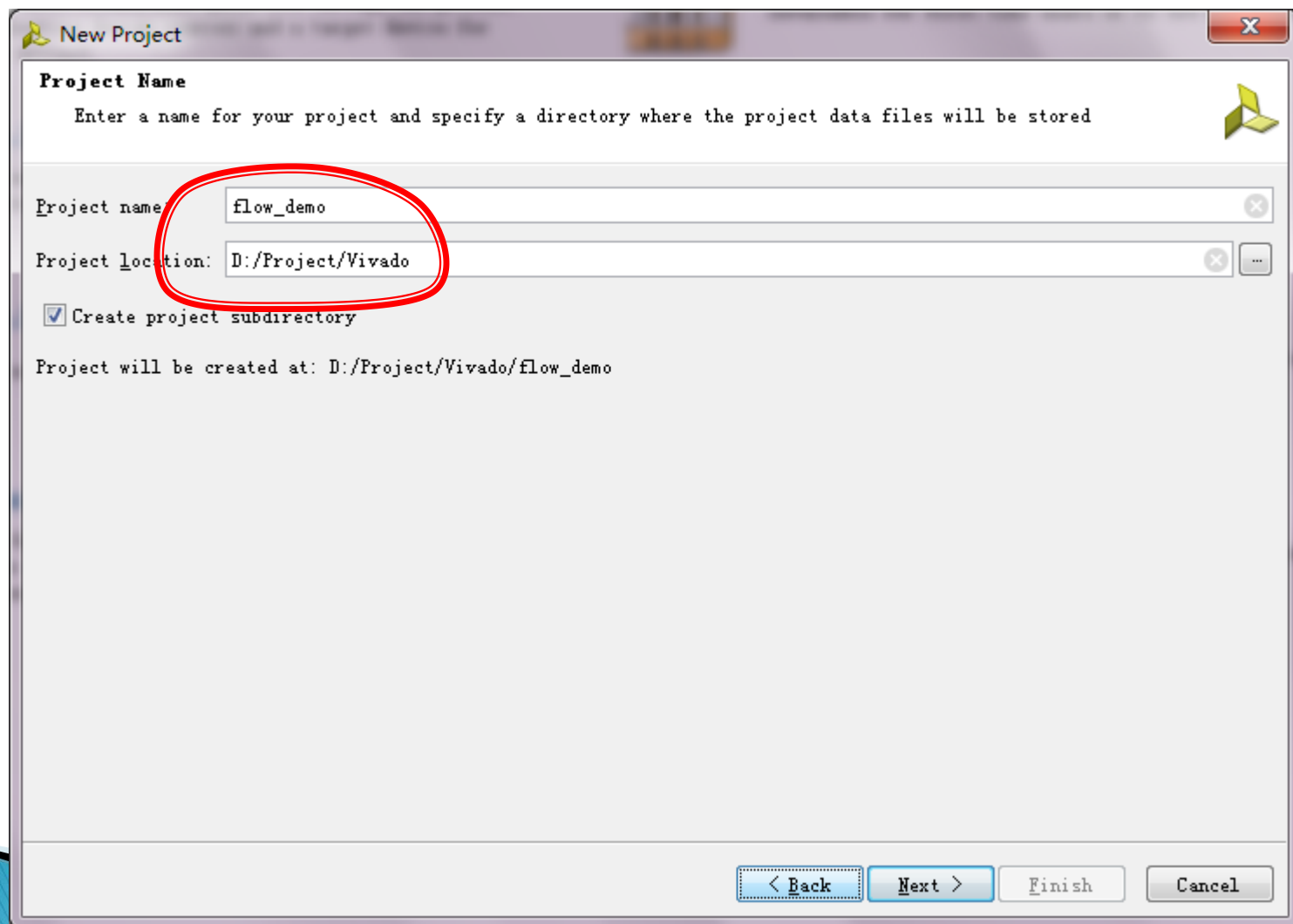
新建工程



新建工程



新建工程



The image shows a 'New Project' dialog box from the Vivado IDE. The dialog has a title bar with the Vivado logo and the text 'New Project'. Inside, the 'Project Name' section is highlighted with a red circle. It contains two text input fields: 'Project name' with the value 'flow_demo' and 'Project location' with the value 'D:/Project/Vivado'. Below these fields is a checked checkbox labeled 'Create project subdirectory'. At the bottom of the dialog, there is a summary line that reads 'Project will be created at: D:/Project/Vivado/flow_demo'. The bottom of the dialog features four buttons: '< Back', 'Next >', 'Finish', and 'Cancel'.

New Project

Enter a name for your project and specify a directory where the project data files will be stored

Project name: flow_demo

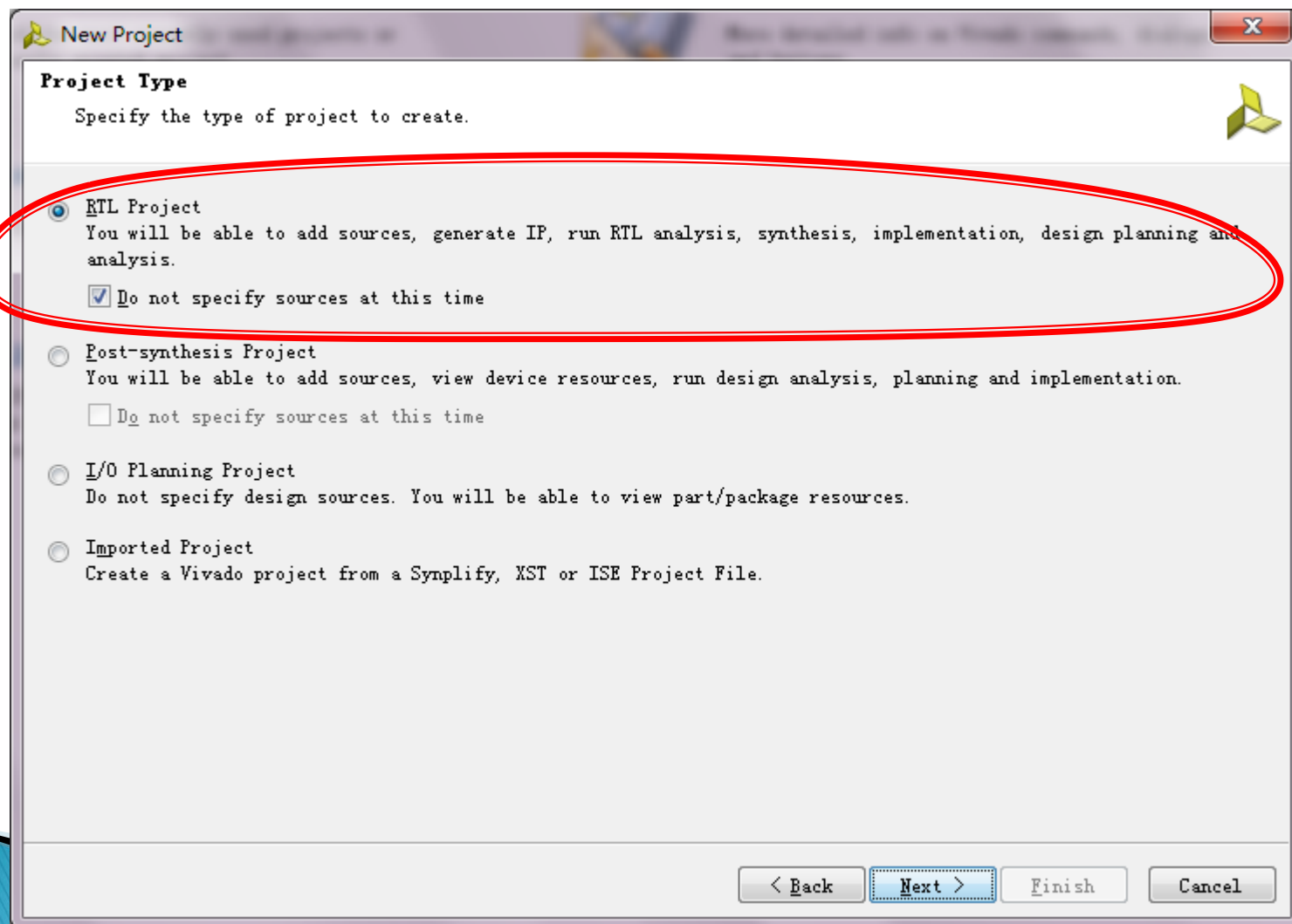
Project location: D:/Project/Vivado

☒ Create project subdirectory

Project will be created at: D:/Project/Vivado/flow_demo

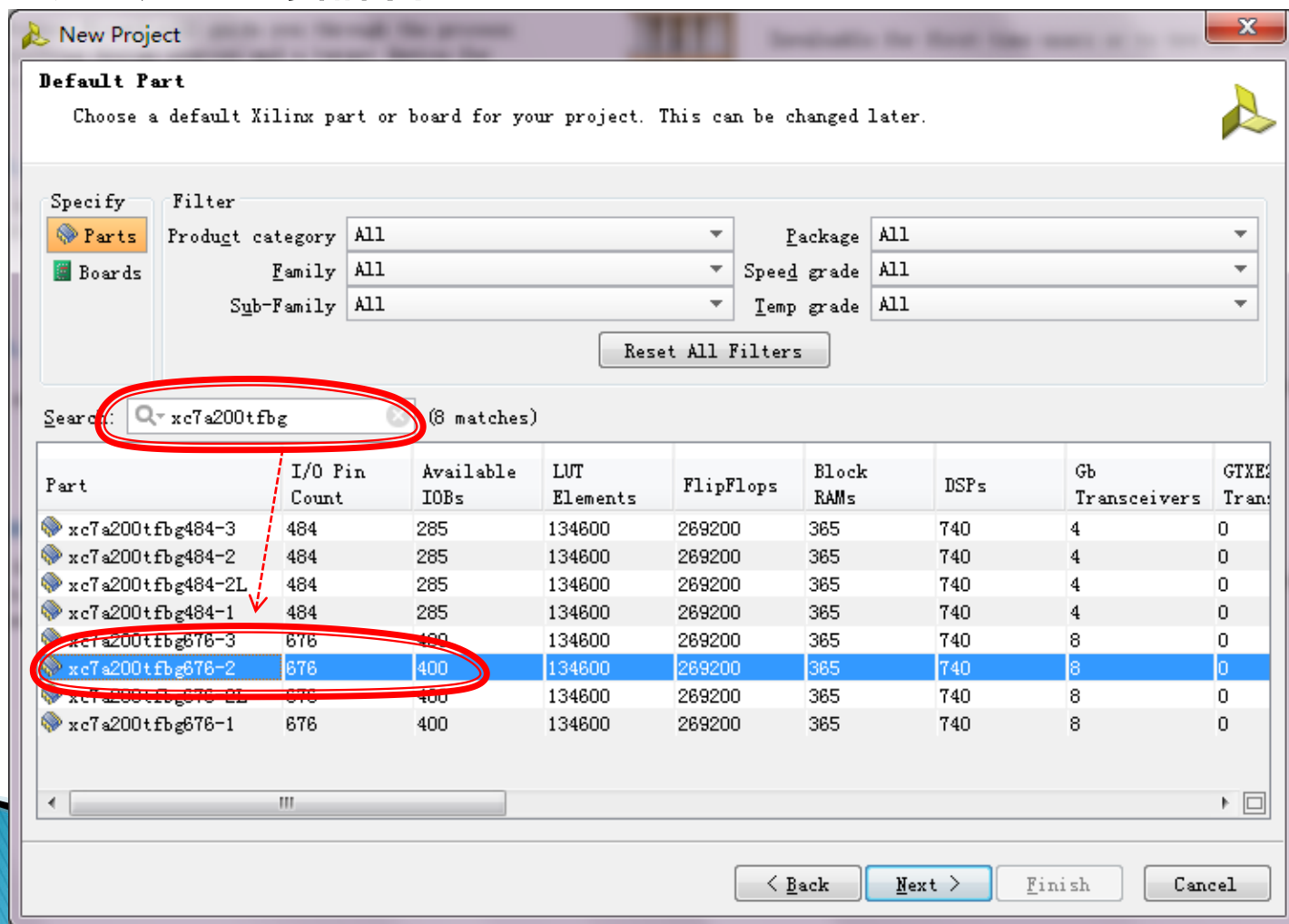
< Back Next > Finish Cancel

新建工程

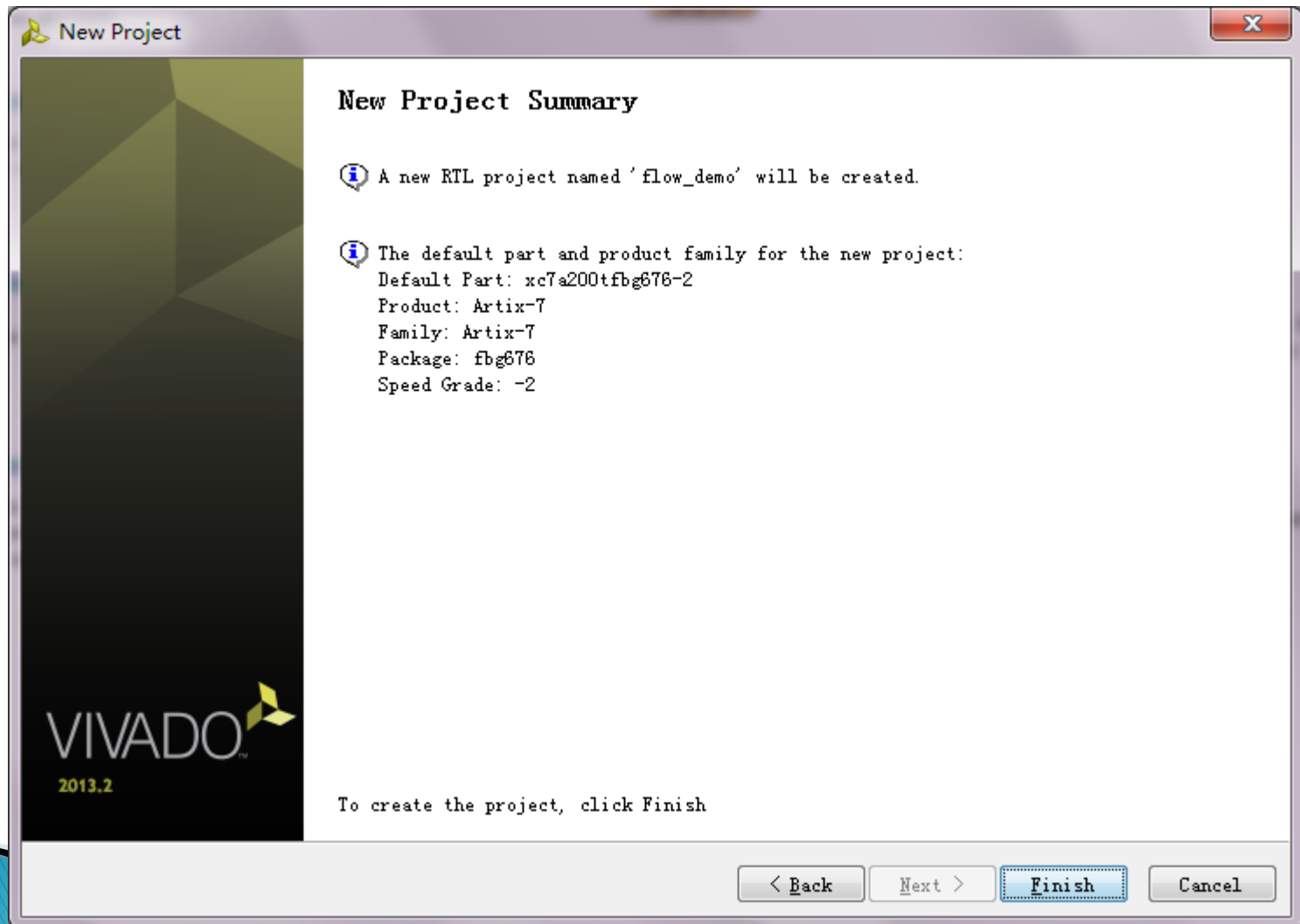


新建工程

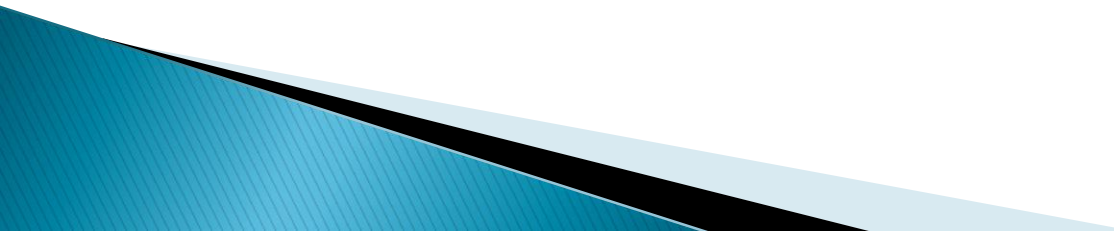
▶ 选定所用的器件



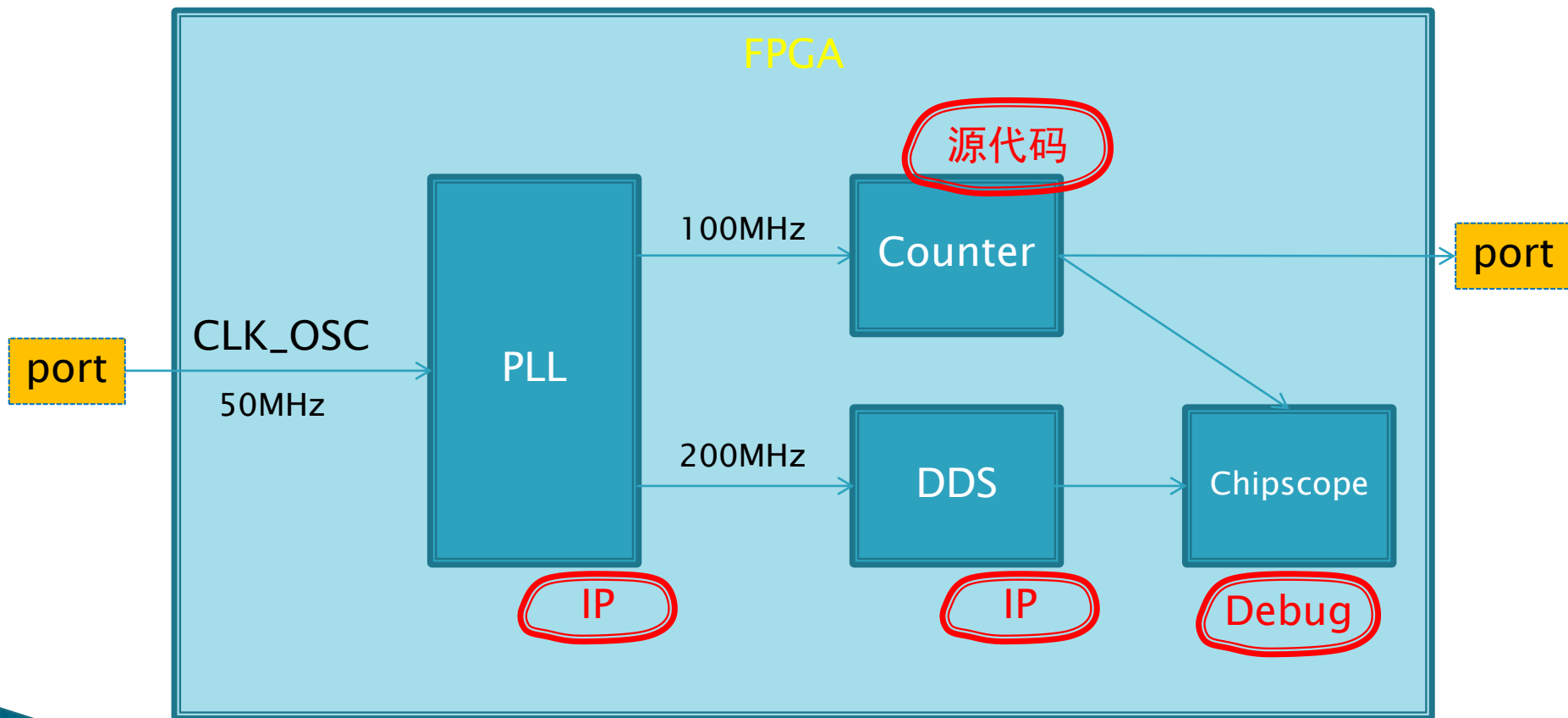
新建工程



示例工程的内容

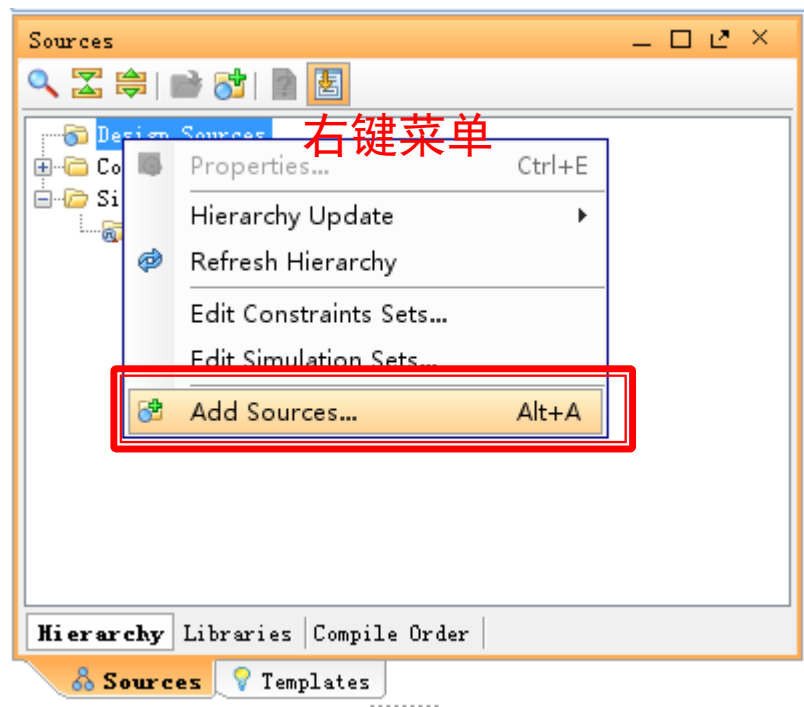
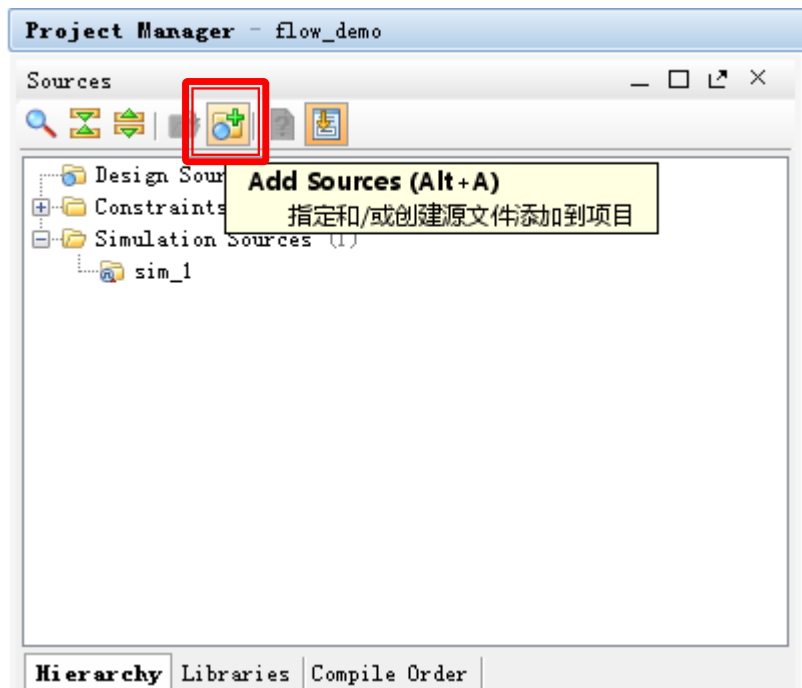
- ▶ 源代码输入
 - ▶ 调用及例化IP
 - ▶ 功能仿真
 - ▶ Chipscope例化
 - ▶ 时钟约束
 - ▶ 管脚锁定
 - ▶ 工程实现
 - ▶ 生成bit文件
- 

示例工程的架构



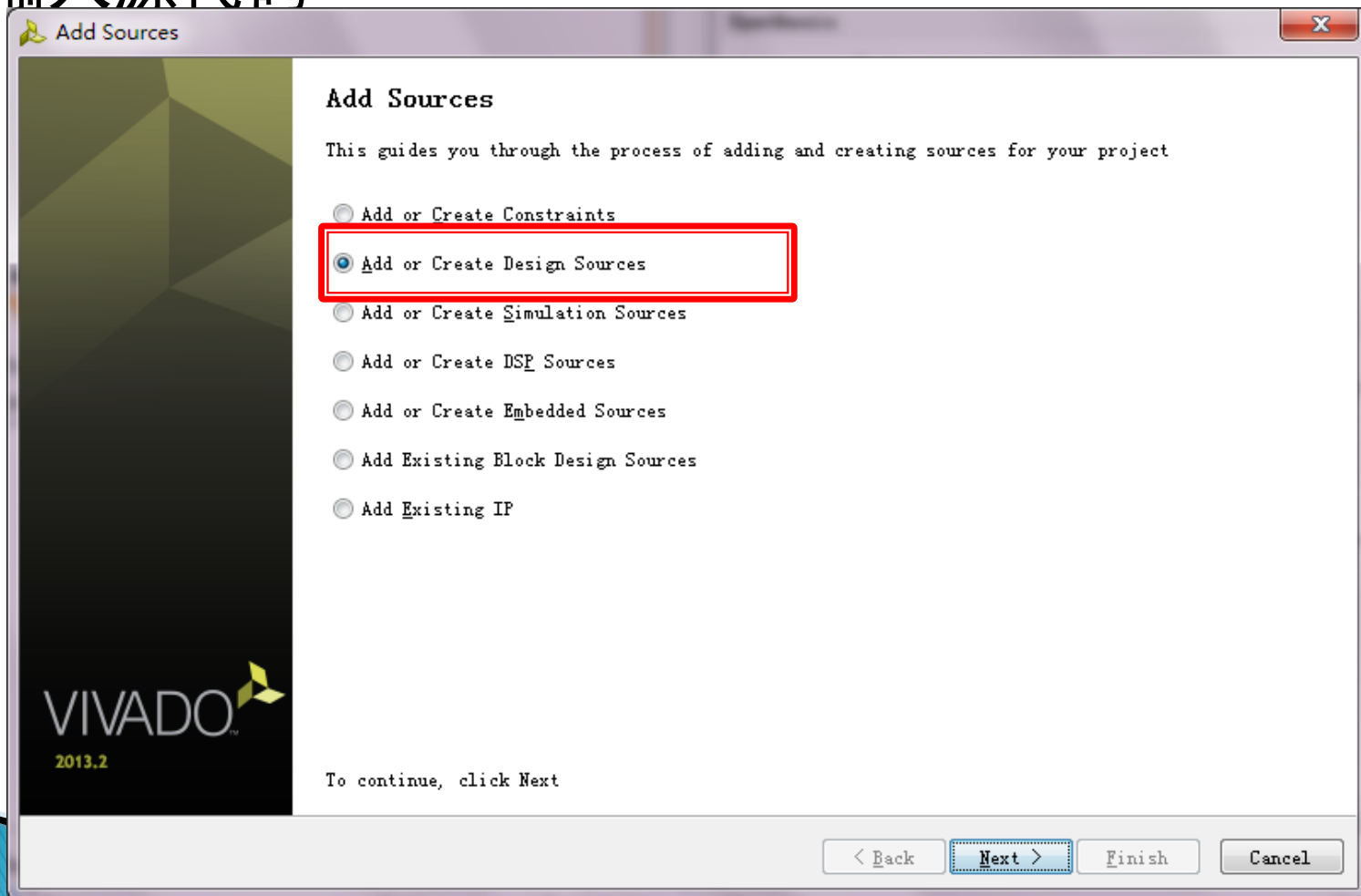
示例工程

► 输入源代码



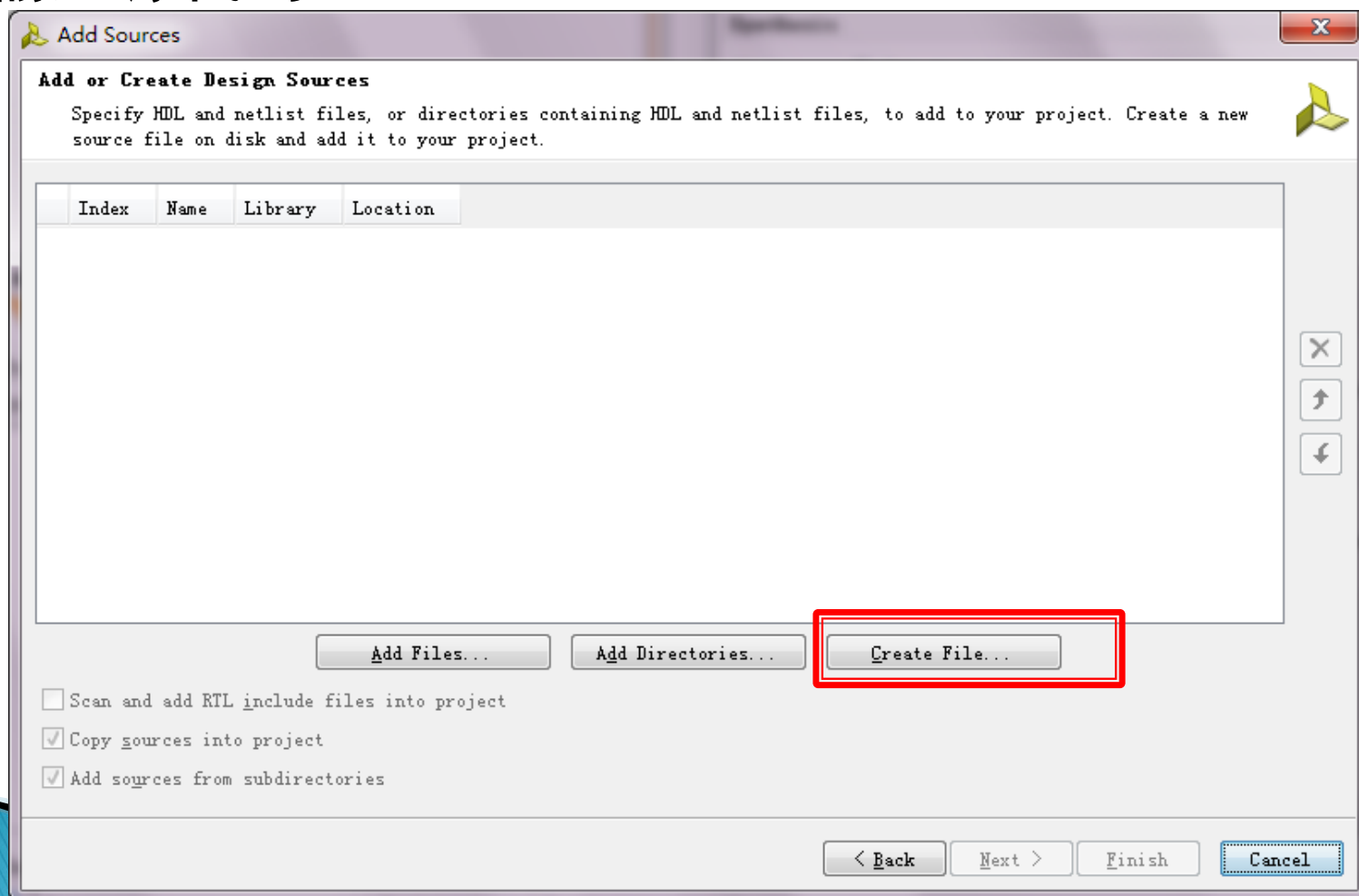
示例工程

▶ 输入源代码



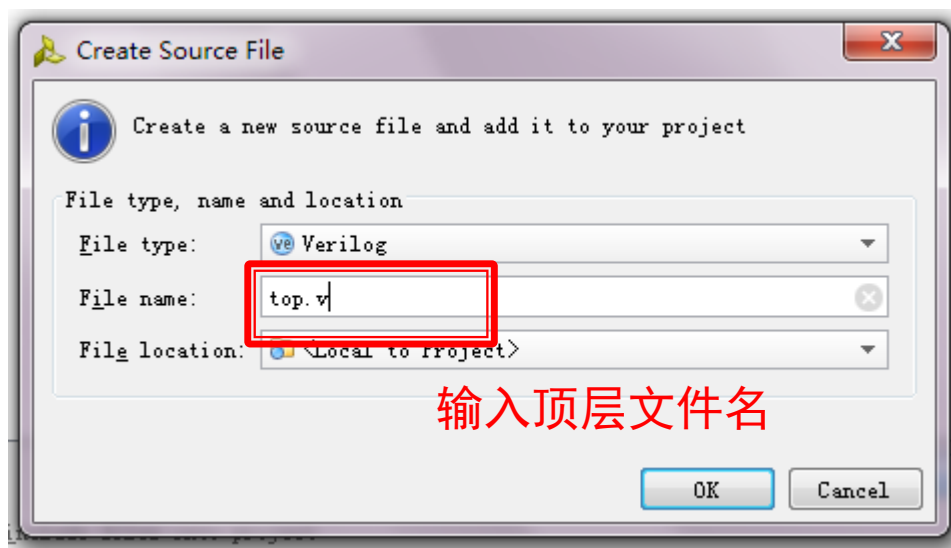
示例工程

▶ 输入源代码



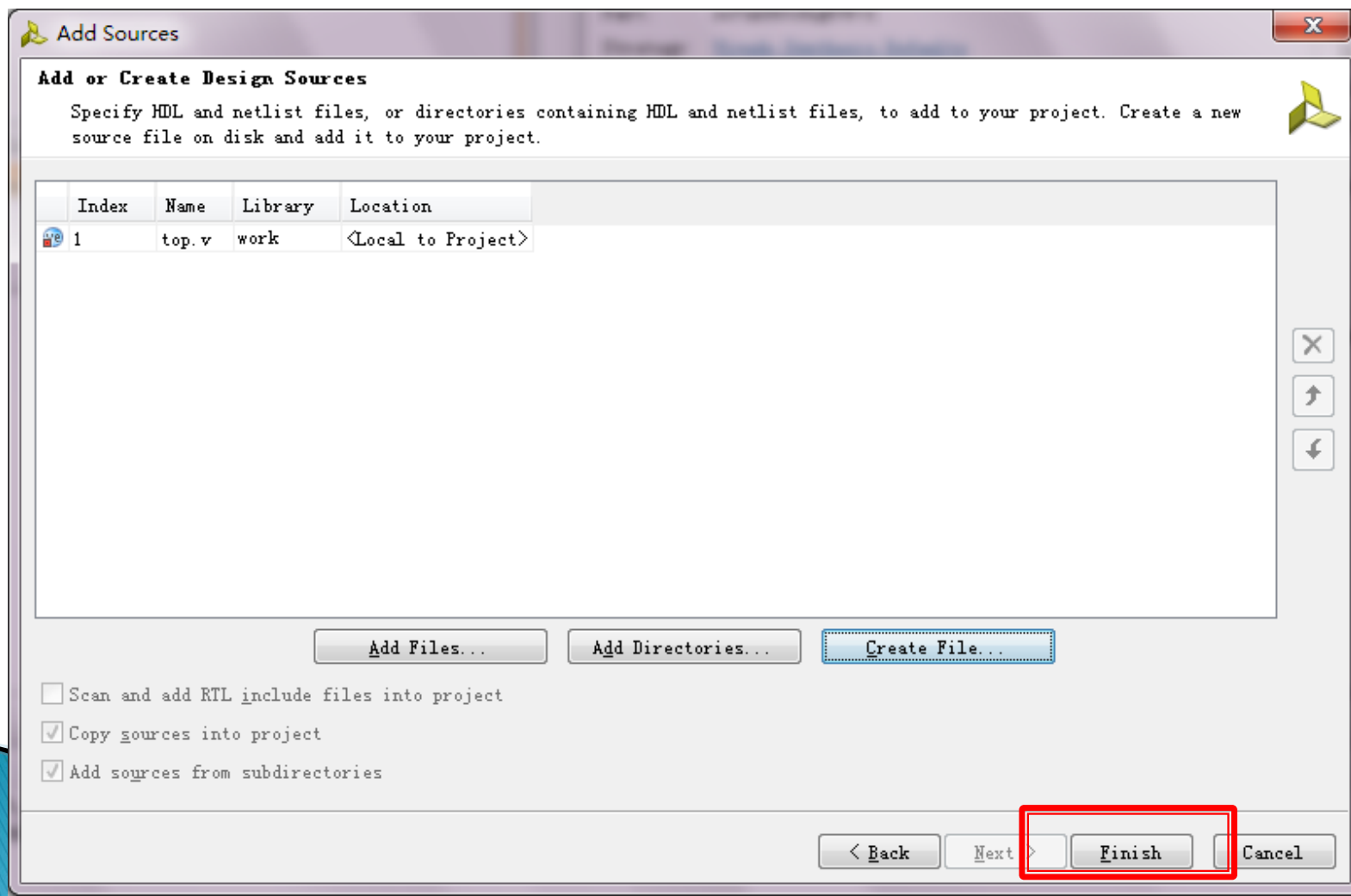
示例工程

► 输入源代码



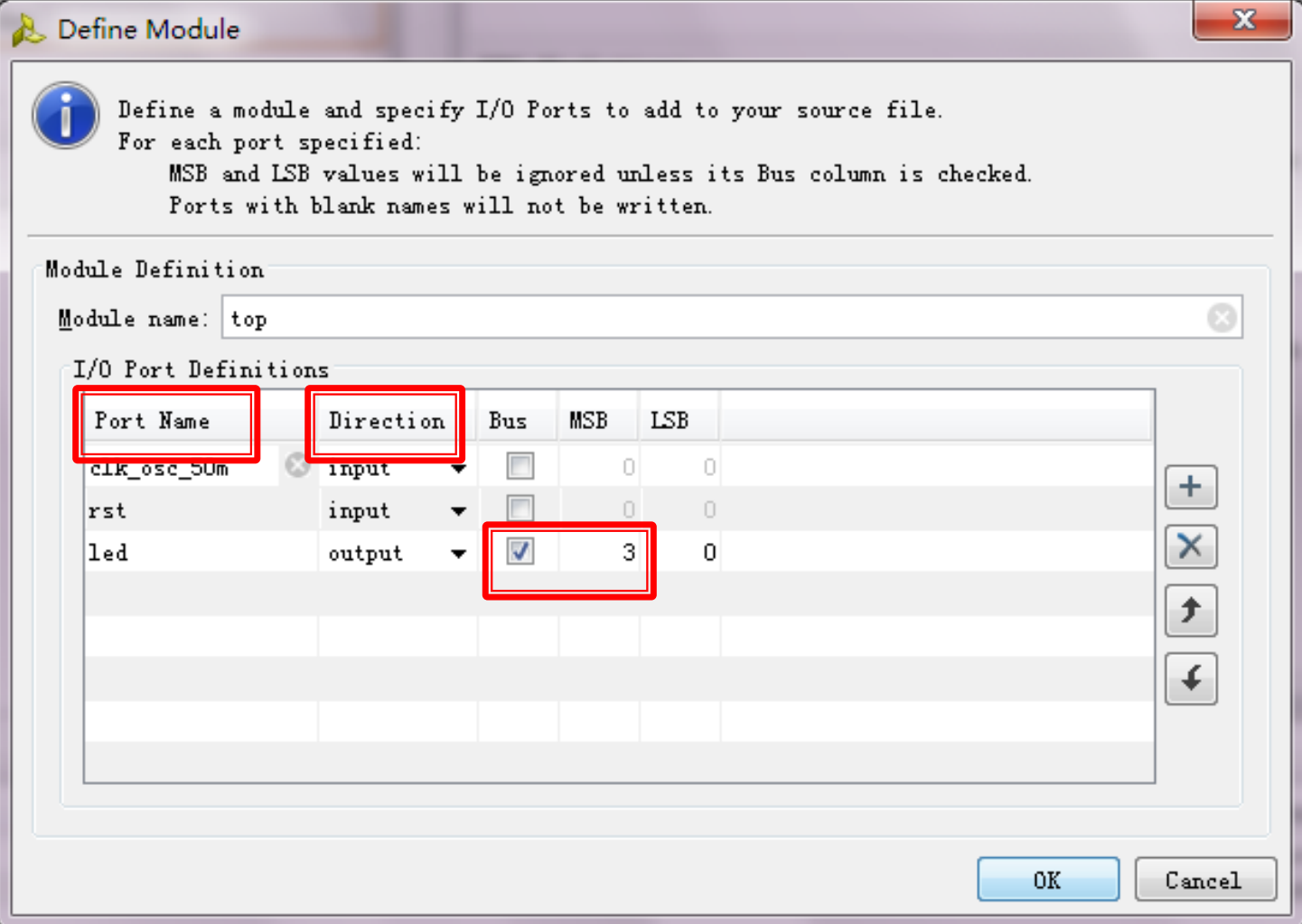
示例工程

► 输入源代码



示例工程

▶ 输入源代码

The image shows a 'Define Module' dialog box in a software development environment. It contains instructions on how to define a module and specify I/O ports. The 'Module name' is set to 'top'. Below, there is a table for 'I/O Port Definitions' with columns for Port Name, Direction, Bus, MSB, and LSB. Three ports are listed: 'clk_osc_50m' (input), 'rst' (input), and 'led' (output). The 'led' port is highlighted with a red box, and its 'Bus' checkbox is checked. The 'Port Name' and 'Direction' headers are also highlighted with red boxes.

Define Module

Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

Module Definition

Module name:

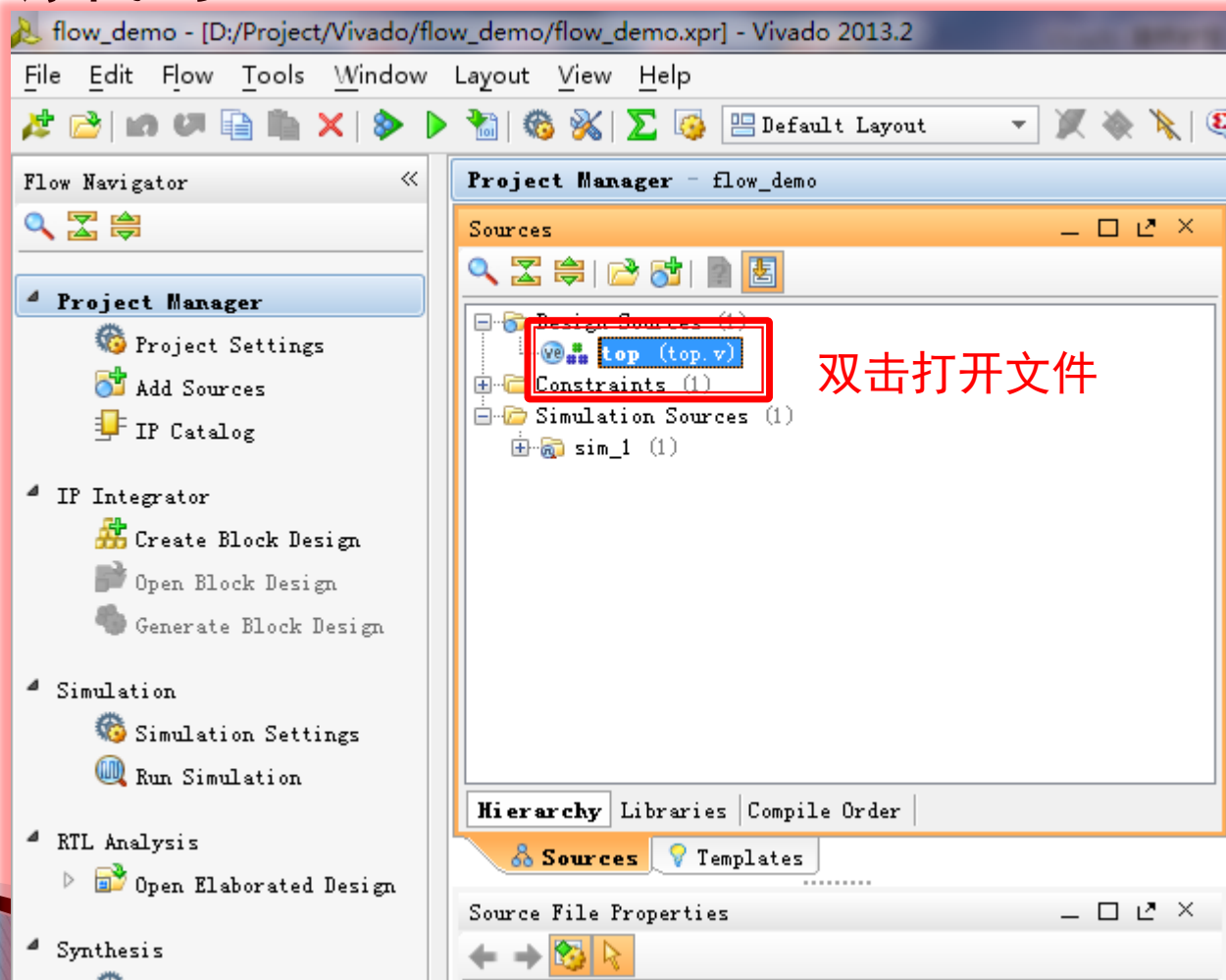
I/O Port Definitions

Port Name	Direction	Bus	MSB	LSB
clk_osc_50m	input	☐	0	0
rst	input	☐	0	0
led	output	☒	3	0

OK Cancel

示例工程

▶ 输入源代码



示例工程

► 输入源代码

The screenshot displays the Vivado Project Manager interface for a project named 'flow_demo'. The 'Sources' pane on the left shows the project hierarchy with 'Design Sources (1)' containing 'top (top.v)'. The 'Source File Properties' pane at the bottom left shows details for 'top.v', including its location, type (Verilog), library (work), size (0.5 KB), and modification date. The 'Project Summary' pane on the right shows the Verilog code for 'top.v', which is a module with two inputs and one output.

Sources

- Design Sources (1)
 - top (top.v)
- Constraints (1)
- Simulation Sources (1)
 - sim_1 (1)

Source File Properties

top.v

Location:	D:/Project/Vivado/flow_demo/srcs/sources_1/new/top.v
Type:	Verilog
Library:	work
Size:	0.5 KB
Modified:	Today at 10:53:
Copied to:	flow_demo.srcs/
Read-only:	No

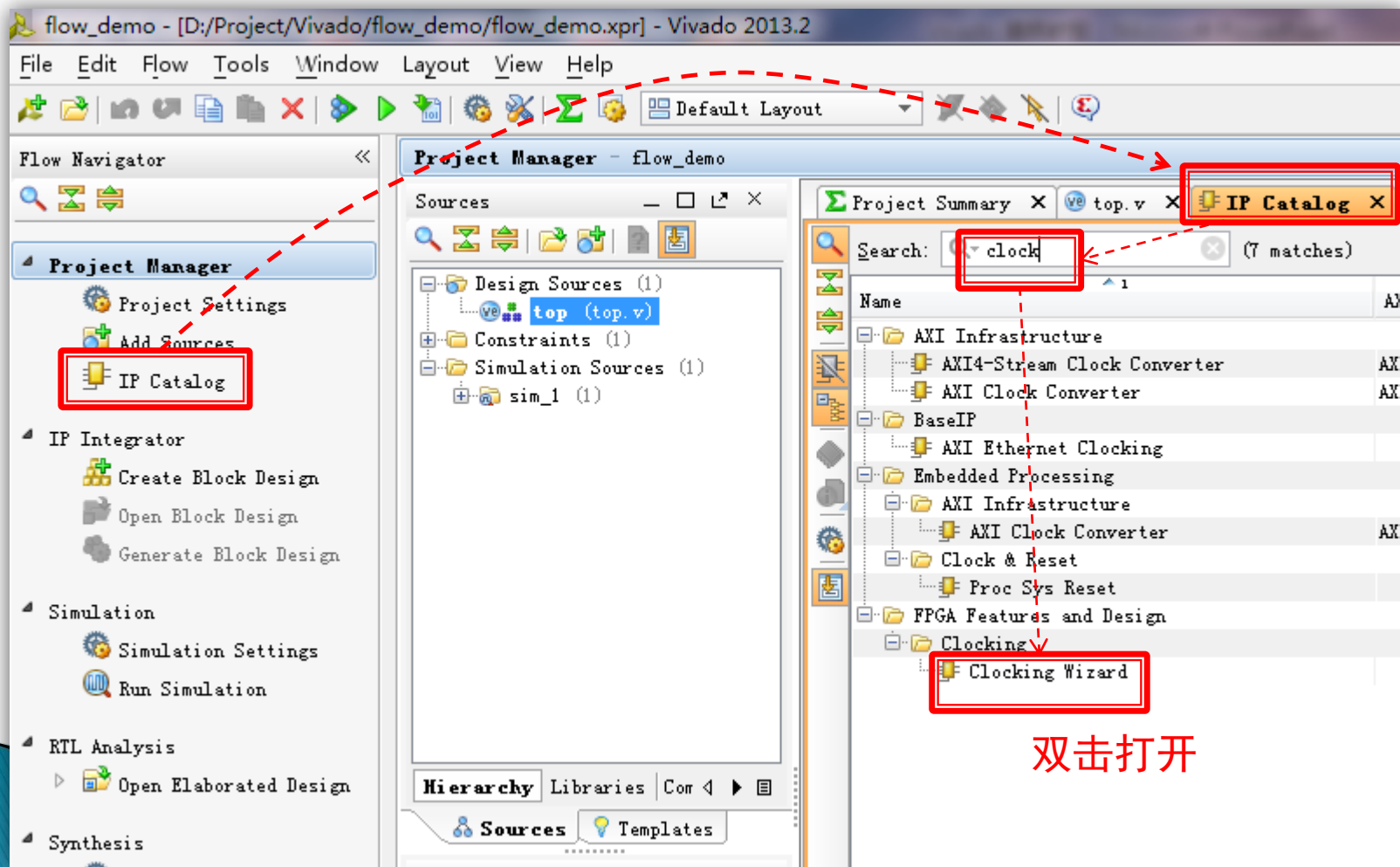
Project Summary

top.v

```
6 // Create Date: 2013/07/23 10:53:28
7 // Design Name:
8 // Module Name: top
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////
21
22
23 module top(
24     input clk_osc_50m,
25     input rst,
26     output [3:0] led
27 );
28 endmodule
29
```

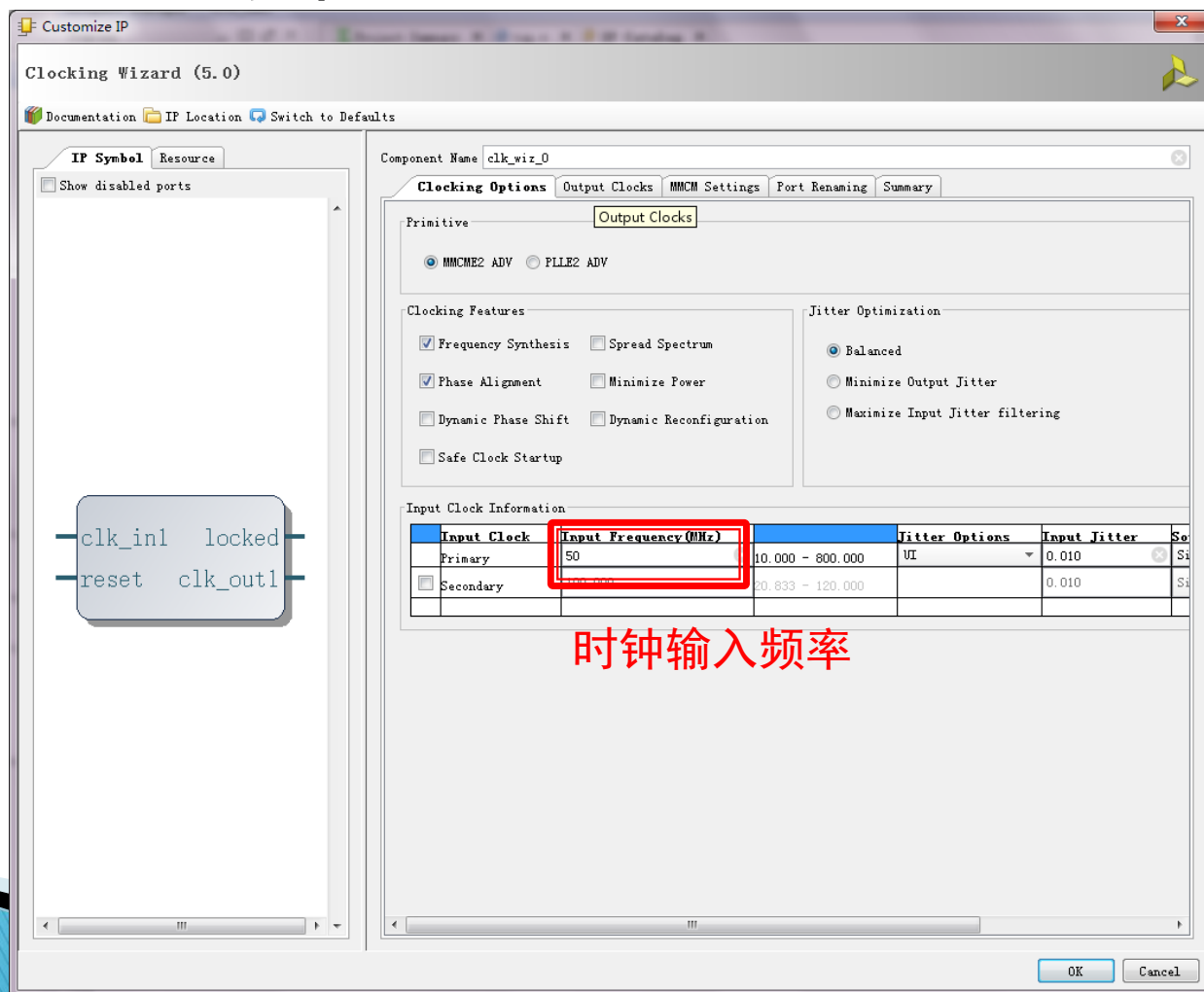
示例工程

▶ 调用IP-PLL时钟



示例工程

▶ 调用IP-PLL时钟



示例工程

► 调用IP-Block

Customize IP

Clocking Wizard (5.0)

Documentation IP Location Switch to Defaults

IP Symbol Resource

Show disabled ports

Component Name: clk_wiz_0

Clocking Options Output Clocks MCM Settings Port Renaming Summary

The following table lists the output clocks relative to the input clock.

Output Clock	Output Freq (MHz)		Phase (degrees)		Duty Cy Request
	Requested	Actual	Requested	Actual	
clk_out1	100.000	100.000	0.000	0.000	50.000
☒ clk_out2	200.000	100.000	0.000	0.000	50.000
☐ clk_out3	100.000	N/A	0.000	N/A	50.000
☐ clk_out4	100.000	N/A	0.000	N/A	50.000
☐ clk_out5	100.000	N/A	0.000	N/A	50.000
☐ clk_out6	100.000	N/A	0.000	N/A	50.000
☐ clk_out7	100.000	N/A	0.000	N/A	50.000

☐ USE CLOCK SEQUENCING

Output Clock	Sequence Number
clk_out1	1
clk_out2	1
clk_out3	1
clk_out4	1
clk_out5	1
clk_out6	1
clk_out7	1

Clocking Feedback

Source

- ☒ Automatic Control On-Chip
- ☐ Automatic Control Off-Chip
- ☐ User-Controlled On-Chip
- ☐ User-Controlled Off-Chip

Signaling

- ☒ Single-ended
- ☐ Differential

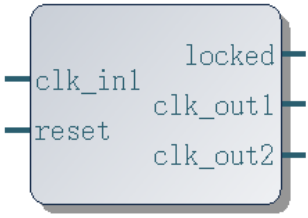
Enable Optional Inputs / Outputs

- ☒ reset ☐ power_down ☐ input_clk_stopped
- ☒ locked ☐ clkfbstopped

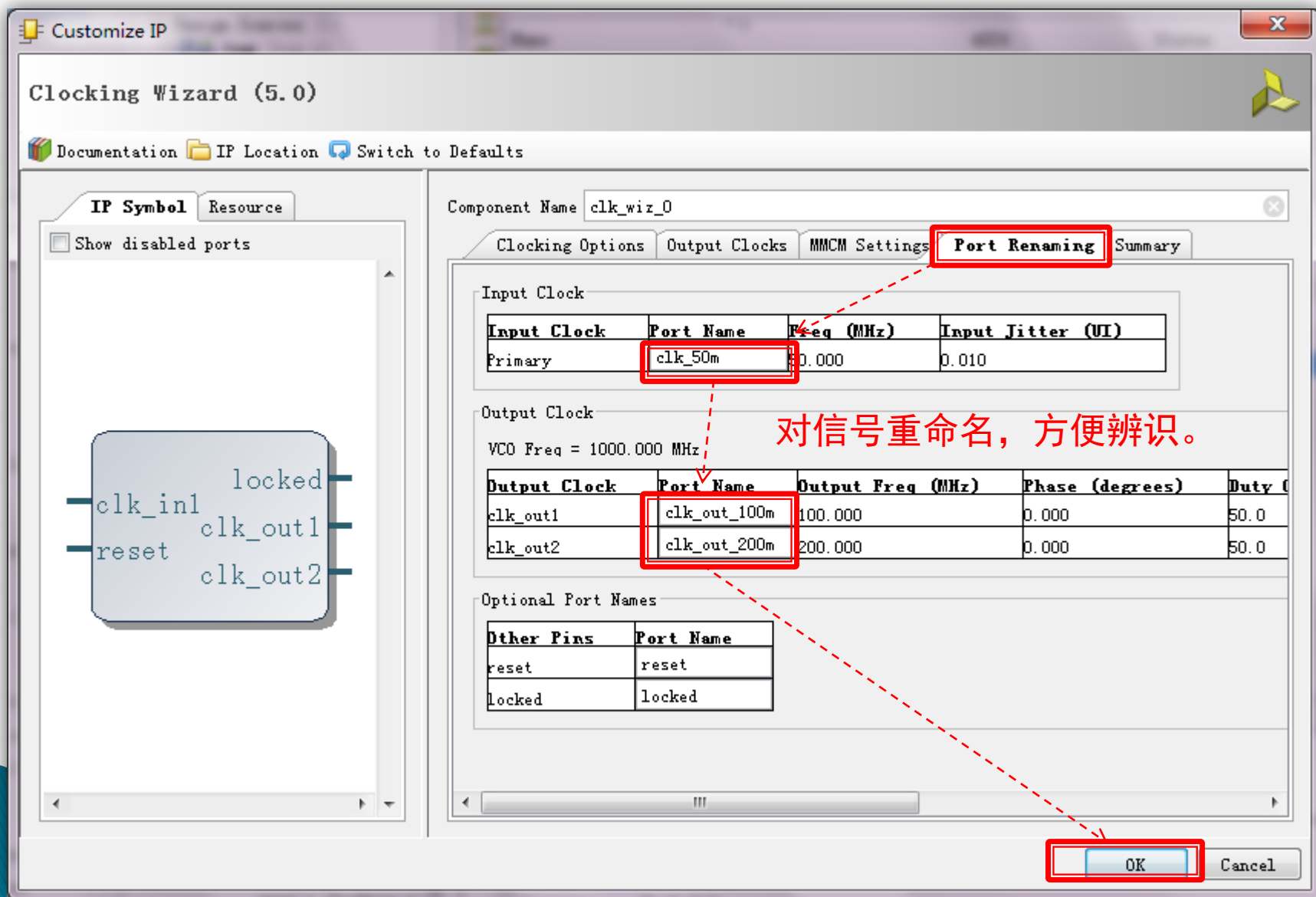
Reset Type

- ☒ Active High
- ☐ Active Low

OK Cancel

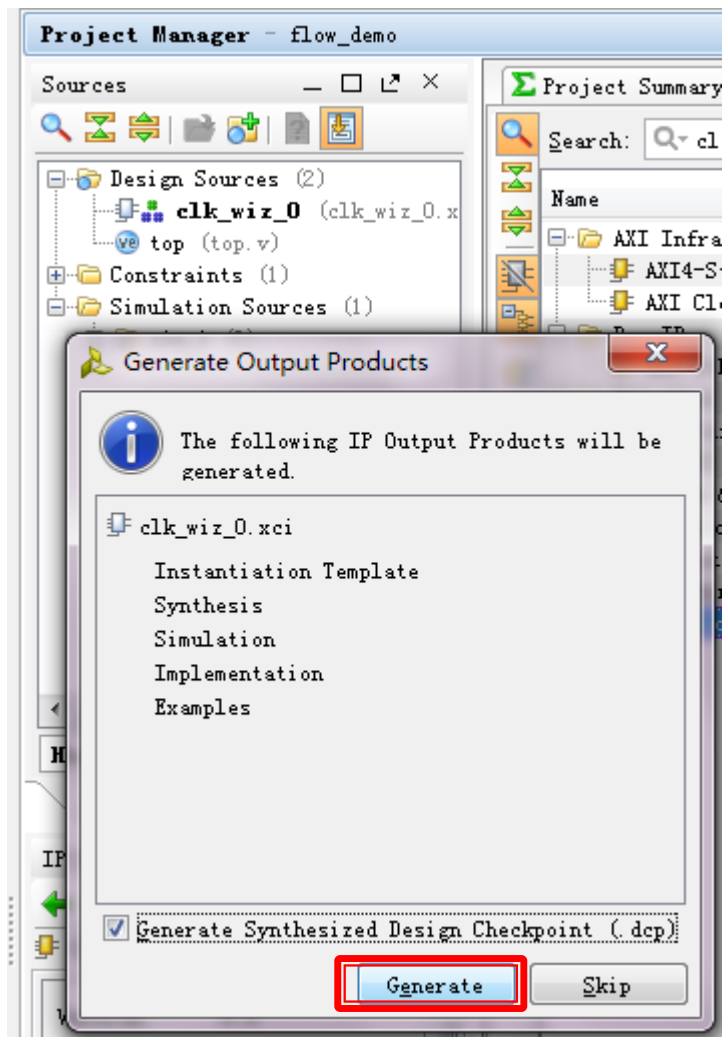


示例工程



示例工程-调用IP

▶ PLL的生成结果



示例工程-调用IP

▶ 时钟 IP例化

The screenshot displays the Vivado Project Manager and Verilog editor. In the Project Manager, the 'IP Sources' tab is selected, showing the 'clk_wiz_0' IP core. A red dashed line connects the 'clk_wiz_0' IP core to the Verilog code in the editor. The Verilog code is a template for instantiating the 'clk_wiz_0' IP core. The code includes comments for input clock, output clocks, and status/control signals. A red box highlights the instantiation code block, and a red arrow points from the 'clk_wiz_0' IP core to this block.

Project Manager - flow_demo

Sources

IP (1)

clk_wiz_0 (13)

clk_wiz_0.v

Simulation (2)

Implementation (1)

Examples (4)

Hierarchy

IP Sources

Libraries

Compile Order

Sources

Templates

Generated Data Properties

clk_wiz_0.v

Location: d:/Project/Vivado/flow_demo/flow_demo.sr

Type: Verilog Template

Size: 3.8 KB

Modified: Today at 13:38:11 PM

Copied to: ./Project/Vivado/flow_demo/flow_demo.sr

Read-only: Yes

Enabled

Project Summary

top.v

IP Catalog

clk_wiz_0.v

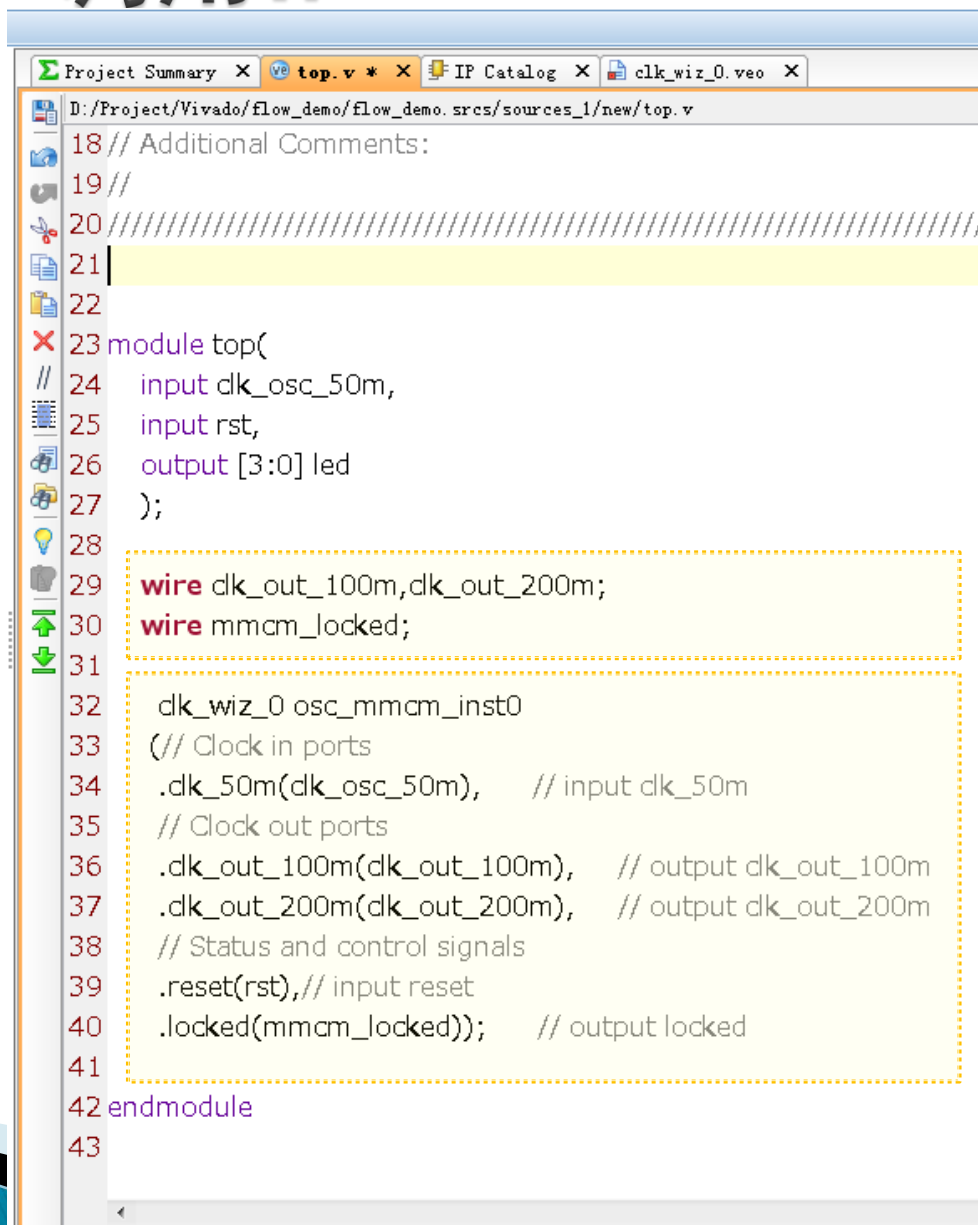
d:/Project/Vivado/flow_demo/flow_demo.sr/srcs/sources_1/ip/clk_wiz_0/clk_wiz_0.v

```
56 //-----
57 // CLK_OUT1__ 100.000 0.000 50.0 162
58 // CLK_OUT2__ 200.000 0.000 50.0 142
59 //
60 //-----
61 // Input Clock Freq (MHz) Input Jitter (UI)
62 //-----
63 // __primary__ 50 0.010
64
65 // The following must be inserted into your Verilog file for this
66 // core to be instantiated. Change the instance name and port c
67 // (in parentheses) to your own signal names.
68
69 //----- Begin Cut here for INSTANTIATION Template ---//
70
71 clk_wiz_0 instance_name
72 (// Clock in ports
73 .dk_50m(dk_50m), // input dk_50m
74 // Clock out ports
75 .dk_out_100m(dk_out_100m), // output dk_out_100m
76 .dk_out_200m(dk_out_200m), // output dk_out_200m
77 // Status and control signals
78 .reset(reset), // input reset
79 .locked(locked)); // output locked
80 // INST_TAG_END ----- End INSTANTIATION Template -----
```

Copy到源代码文件里

示例工程-调用IP

▶ 时钟 IP例化

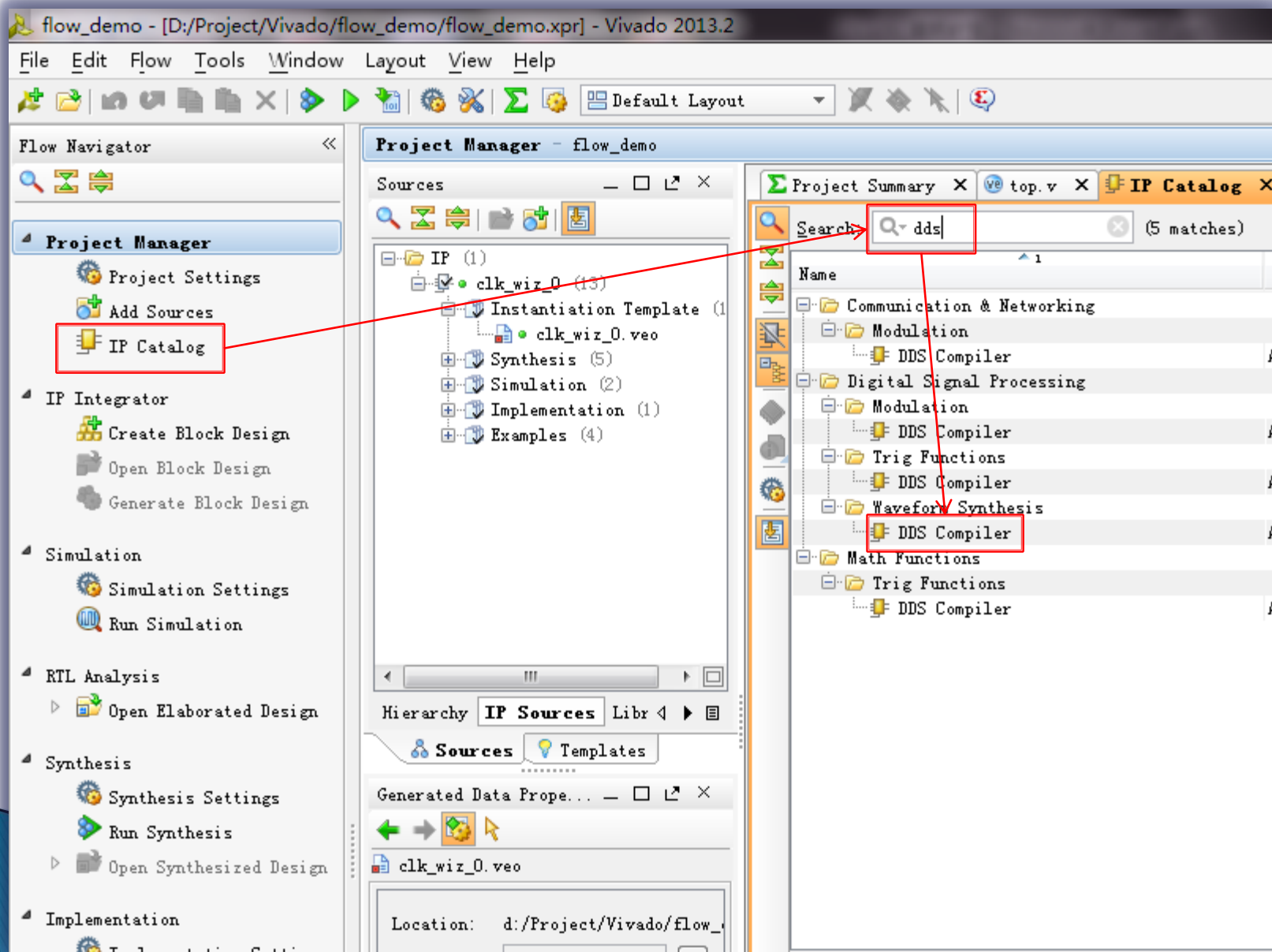


```
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module top(
24     input clk_osc_50m,
25     input rst,
26     output [3:0] led
27 );
28
29     wire clk_out_100m, clk_out_200m;
30     wire mmcm_locked;
31
32     clk_wiz_0 osc_mmcm_inst0
33     (// Clock in ports
34     .clk_50m(clk_osc_50m),    // input clk_50m
35     // Clock out ports
36     .clk_out_100m(clk_out_100m),    // output clk_out_100m
37     .clk_out_200m(clk_out_200m),    // output clk_out_200m
38     // Status and control signals
39     .reset(rst), // input reset
40     .locked(mmcm_locked));    // output locked
41
42 endmodule
43
```

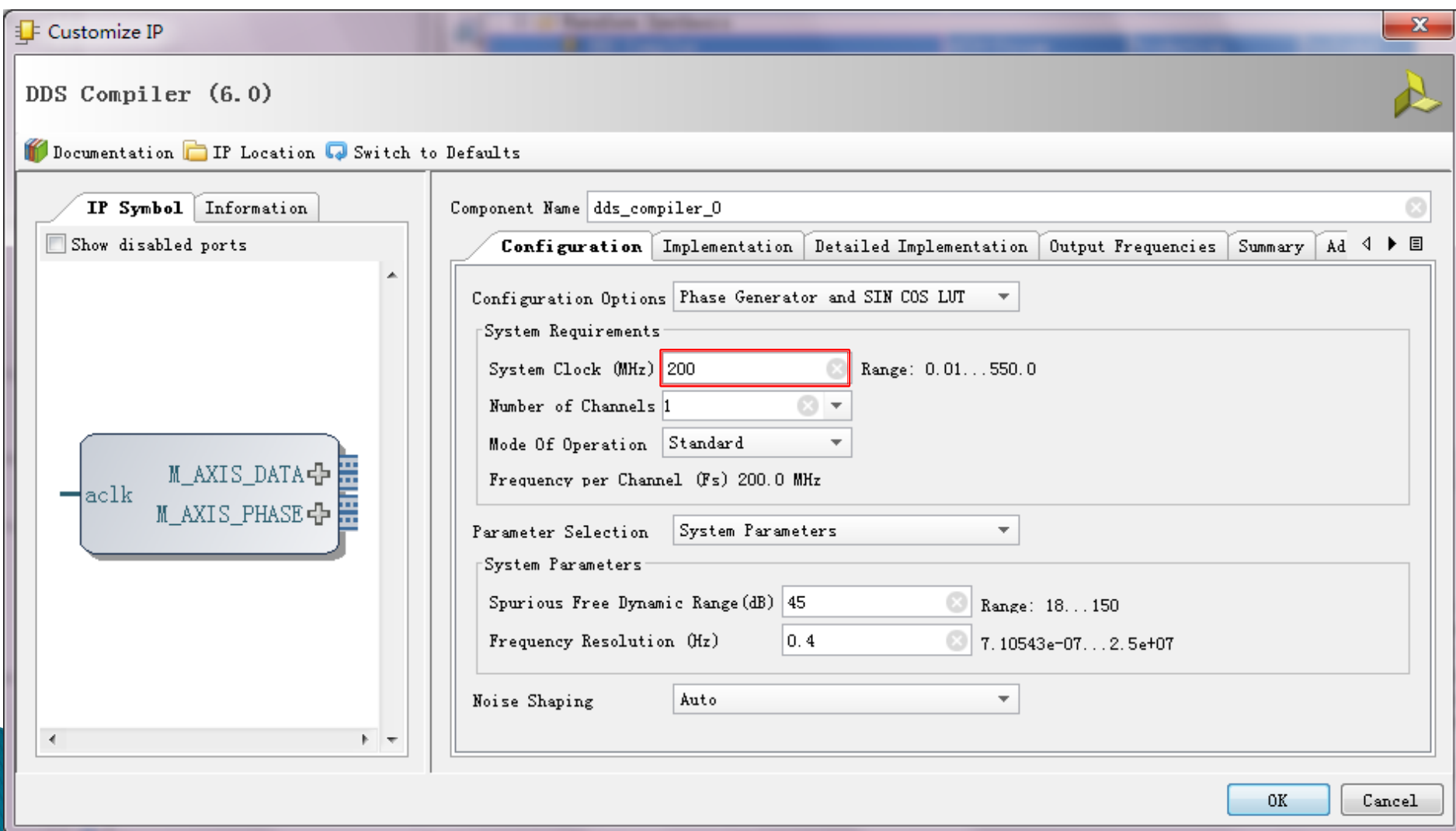
信号声明

IP例化

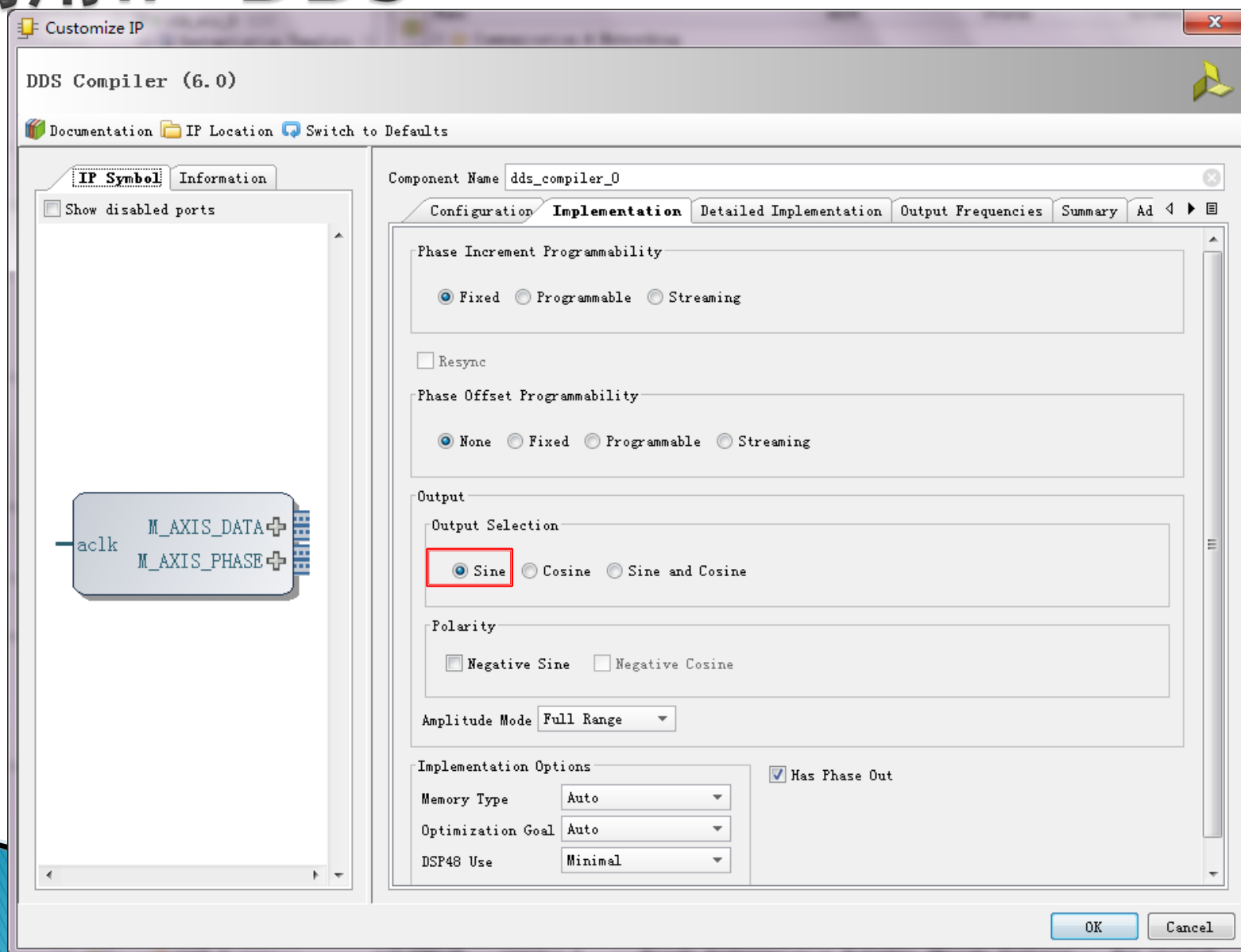
调用IP-DDS



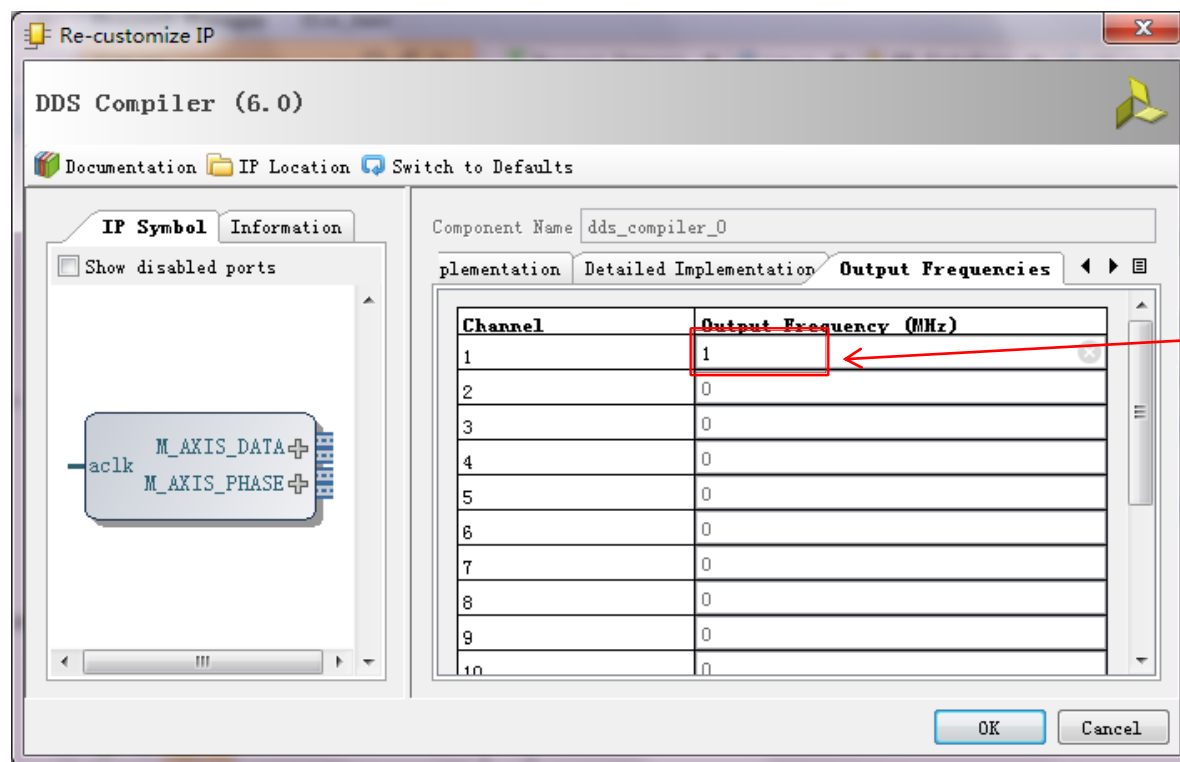
调用IP-DDS



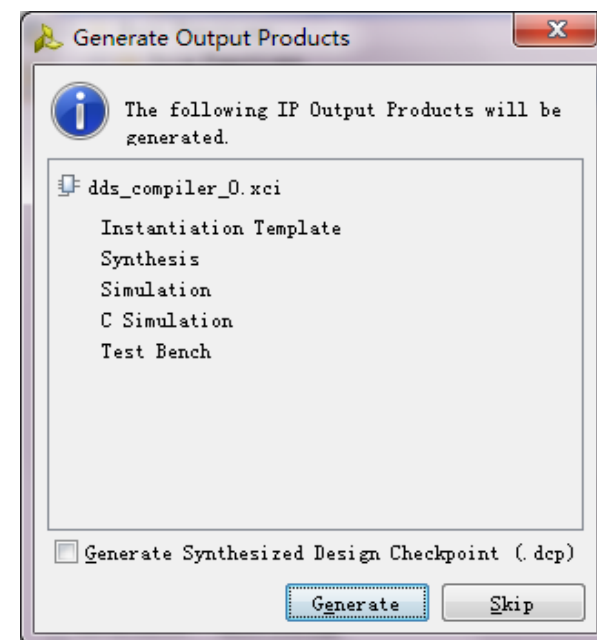
调用IP-DDS



调用IP-DDS



输入需要的频率



调用IP-DDS

The screenshot displays the Vivado Project Manager interface for a project named 'flow_demo'. The 'Sources' pane on the left shows a tree structure under 'IP (2)' containing 'dds_compiler_0 (8)'. Within this, 'Instantiation Template (1)' contains 'dds_compiler_0.vao', which is highlighted with a red box. A red arrow points from this box to the 'IP Sources' tab in the 'Hierarchy' pane. Below this, the 'Generated Data Properties' pane shows 'dds_compiler_0.vao' with a 'Location' of 'd:/Project/Vivado/flow_demo' and a 'Type' of 'Verilog Template'.

The main editor window shows the Verilog code for 'dds_compiler_0.vao'. The code includes comments and instantiation logic. A red box highlights the instantiation code block, which is connected by a red arrow from the 'Instantiation Template' entry in the project tree.

```
45 // PART OF THIS FILE AT ALL TIMES.
46 //
47 // DO NOT MODIFY THIS FILE.
48
49 // IP VLNV: xilinx.com:ip:dds_compiler:6.0
50 // IP Revision: 1
51
52 // The following must be inserted into your Verilog
53 // core to be instantiated. Change the instance name
54 // (in parentheses) to your own signal names.
55
56 //----- Begin Cut Here for INSTANTIATION Template -----
57 dds_compiler_0 your_instance_name (
58     .adk(adk), // input adk
59     .m_axis_data_tvalid(m_axis_data_tvalid), // output m_axis_data_tvalid
60     .m_axis_data_tdata(m_axis_data_tdata), // output m_axis_data_tdata
61     .m_axis_phase_tvalid(m_axis_phase_tvalid), // output m_axis_phase_tvalid
62     .m_axis_phase_tdata(m_axis_phase_tdata) // output m_axis_phase_tdata
63 );
64 // INST_TAG_END ----- End INSTANTIATION Template -----
```

调用IP-DDS

```
Project Summary x dds_compiler_0_vco x top.v * x
D:/Project/Vivado/flow_demo/flow_demo.srcs/sources_1/new/top.v
30 wire mmcm_locked;
31
32 dk_wiz_0 osc_mmcm_inst0
33 (// Clock in ports
34 .clk_50m(dk_osc_50m), // input clk_50m
35 // Clock out ports
36 .clk_out_100m(dk_out_100m), // output clk_out_100m
37 .clk_out_200m(dk_out_200m), // output clk_out_200m
38 // Status and control signals
39 .reset(rst), // input reset
40 .locked(mmcm_locked)); // output locked
41
42 wire m_axis_data_tvalid, m_axis_phase_tvalid;
43 wire[7:0] m_axis_data_tdata;
44 wire[31:0] m_axis_phase_tdata;
45 dds_compiler_0 dds_inst0 (
46 .adk(dk_out_200m), // input adk
47 .m_axis_data_tvalid(m_axis_data_tvalid), // output m_axis_data_tvalid
48 .m_axis_data_tdata(m_axis_data_tdata), // output [7 : 0] m_axis_data_tdata
49 .m_axis_phase_tvalid(m_axis_phase_tvalid), // output m_axis_phase_tvalid
50 .m_axis_phase_tdata(m_axis_phase_tdata) // output [31 : 0] m_axis_phase_tdata
51 );
52
53 endmodule
```

信号声明

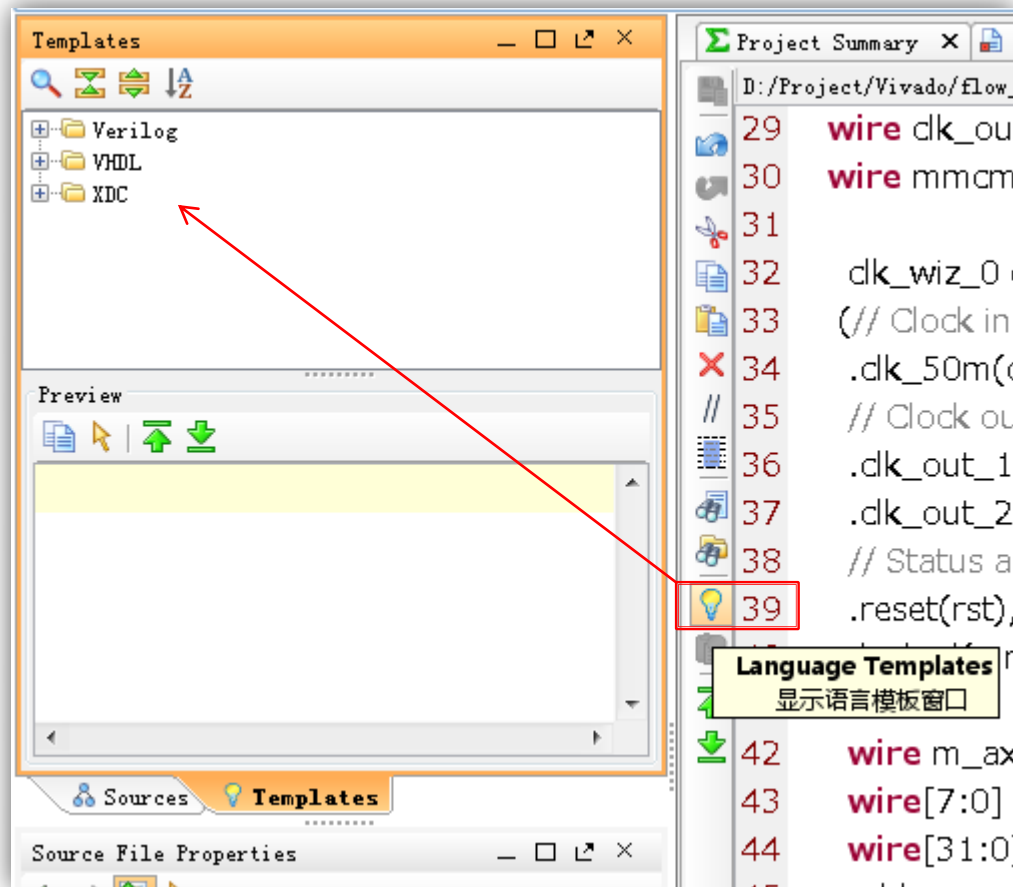
IP例化

Debug变量声明

- ▶ 在插入Chipscope ILA模块时，可以直接找到Debug变量。

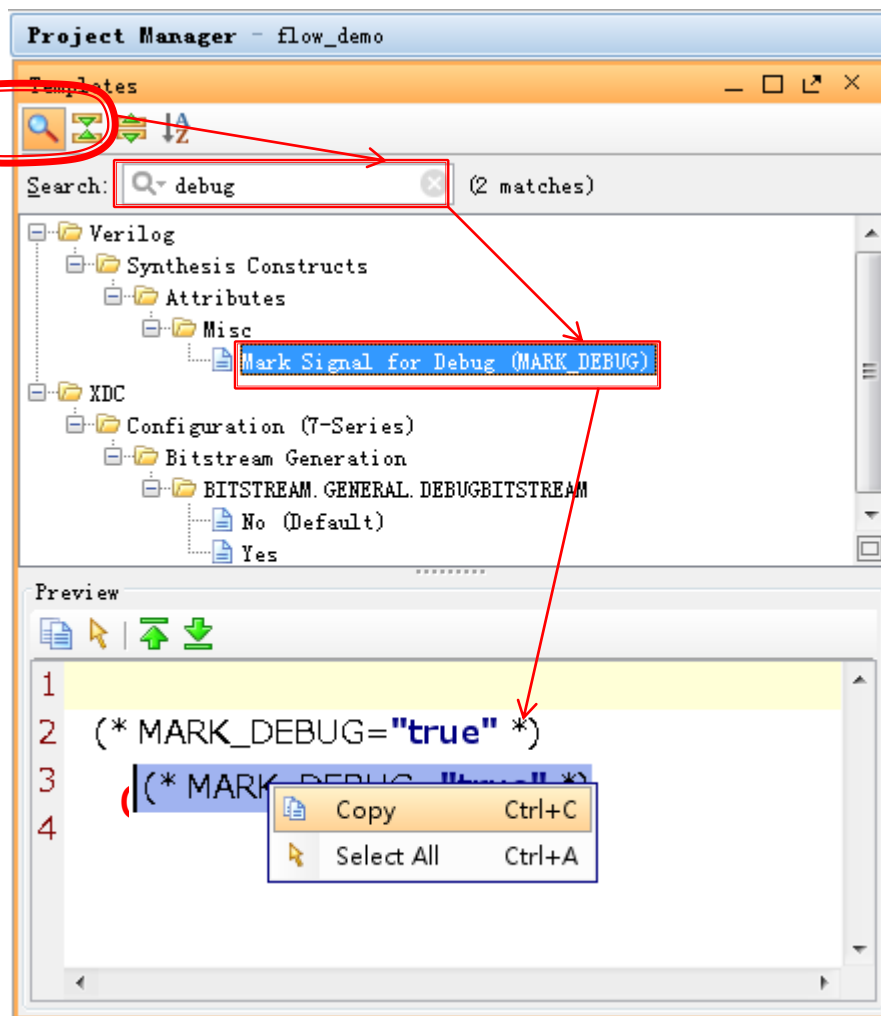
Debug变量声明

▶ 启动代码模板



Debug变量声明

- ▶ 搜索“debug”
关键字



Debug变量声明

- ▶ 声明为“DEBUG”，即使没有连接到其他模块，也不会被优化掉。

```
D:/Project/Vivado/flow_demo/flow_demo.srcs/sources_1/new/top.v
32  clk_wiz_0 osc_mmcm_inst0
33  (// Clock in ports
34  .clk_50m(dk_osc_50m),    // input dk_50m
35  // Clock out ports
36  .clk_out_100m(dk_out_100m), // output dk_out_100m
37  .clk_out_200m(dk_out_200m), // output dk_out_200m
38  // Status and control signals
39  .reset(rst), // input reset
40  .locked(mmcm_locked)); // output locked
41
42  wire m_axis_data_tvalid, m_axis_phase_tvalid;
43  wire[7:0] m_axis_data_tdata;
44  wire[31:0] m_axis_phase_tdata;
45  dds_compiler_0 dds_inst0 (
46  .adk(dk_out_200m), // input adk
47  .m_axis_data_tvalid(m_axis_data_tvalid), // output m_axis_data
48  .m_axis_data_tdata(m_axis_data_tdata), // output [7 : 0] m_ax
49  .m_axis_phase_tvalid(m_axis_phase_tvalid), // output m_axis_p
50  .m_axis_phase_tdata(m_axis_phase_tdata) // output [31 : 0] m
51  );
52
53  (* MARK_DEBUG="true" *) reg[7:0] m_axis_data_tdata_r;
54  (* MARK_DEBUG="true" *) reg[31:0] m_axis_phase_tdata_r;
55
56
57
58 endmodule
```

DDS模块

▶ DEBUG变量的实现

```
52  
53 (* MARK_DEBUG="true" *) reg[7:0] m_axis_data_tdata_r;  
54 (* MARK_DEBUG="true" *) reg[31:0] m_axis_phase_tdata_r;  
55  
56 always @(posedge clk_out_200m)  
57 begin  
58     if(m_axis_data_tvalid)  
59         m_axis_data_tdata_r<=m_axis_data_tdata;  
60 end  
61  
62 always @(posedge clk_out_200m)|  
63 begin  
64     if(m_axis_phase_tvalid)  
65         m_axis_phase_tdata_r<=m_axis_phase_tdata;  
66 end  
67  
68 endmodule
```

Counter模块实现

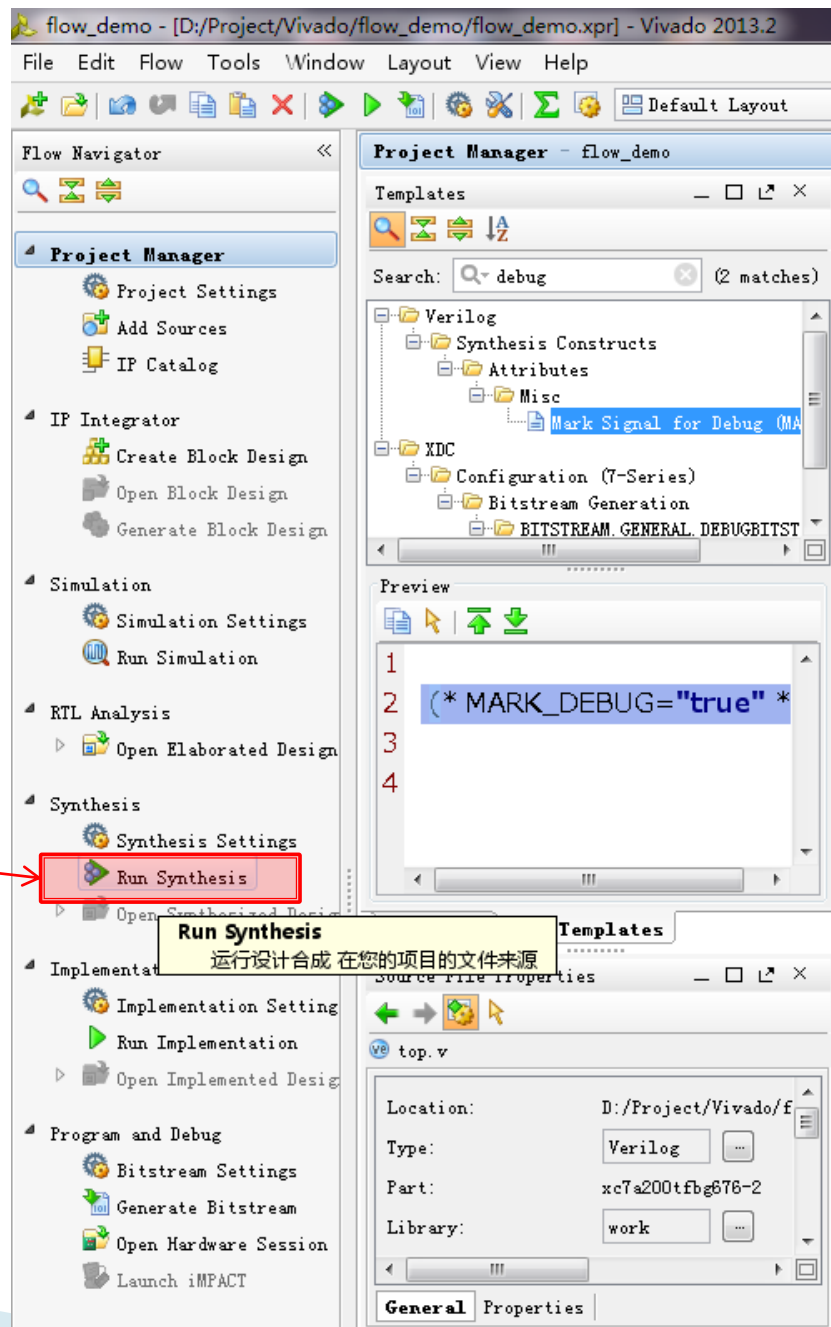
```
68 (* MARK_DEBUG="true" *) reg[23:0] led_r=0;
```

声明时赋初始值；
不要使用reset赋值方式！

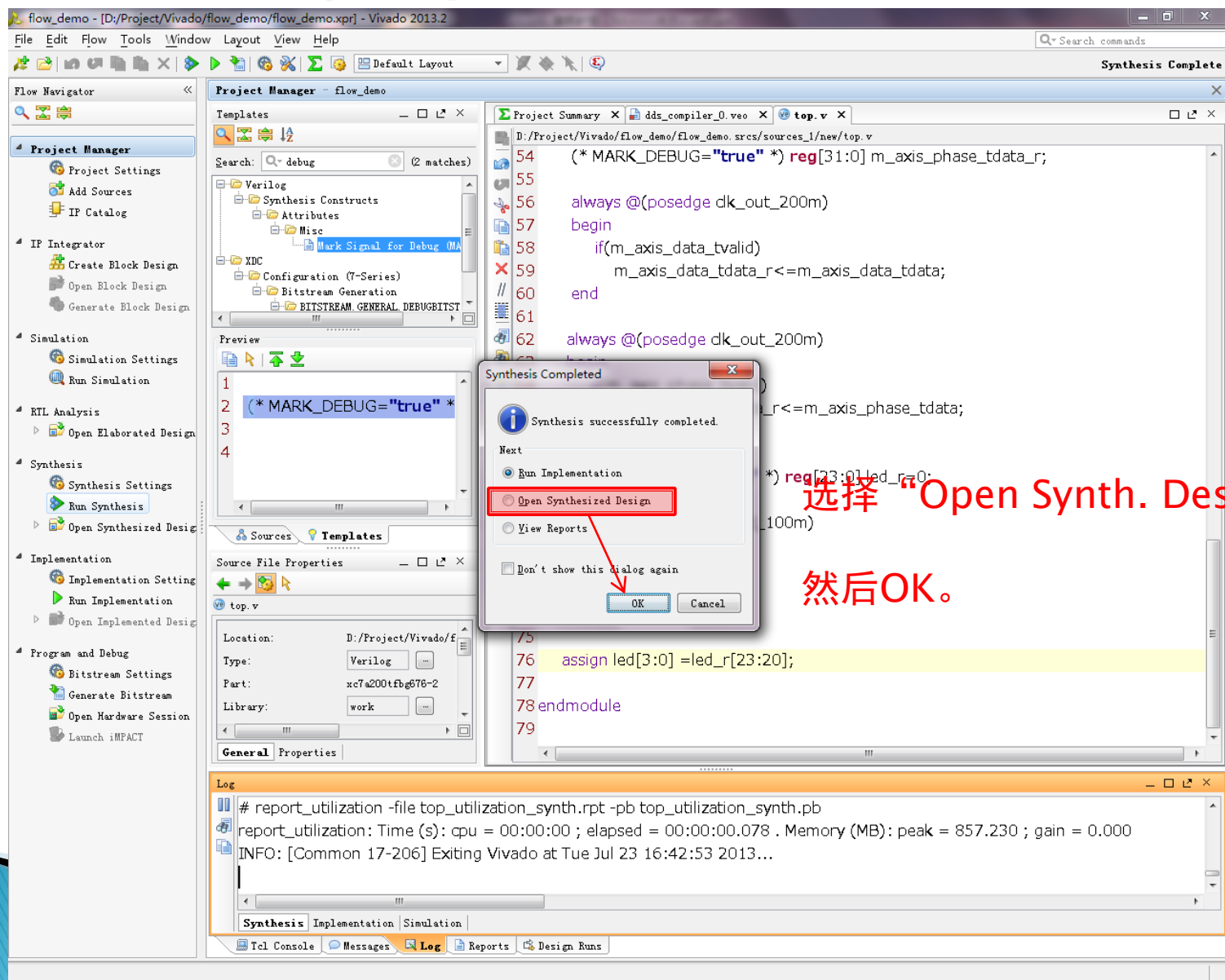
```
69  
70 always @(posedge clk_out_100m)  
71 begin  
72     if(mmc_m_locked)  
73         led_r<=led_r + 1;  
74 end  
75  
76 assign led[3:0] =led_r[23:20];  
77  
78 endmodule
```

Synthesis!

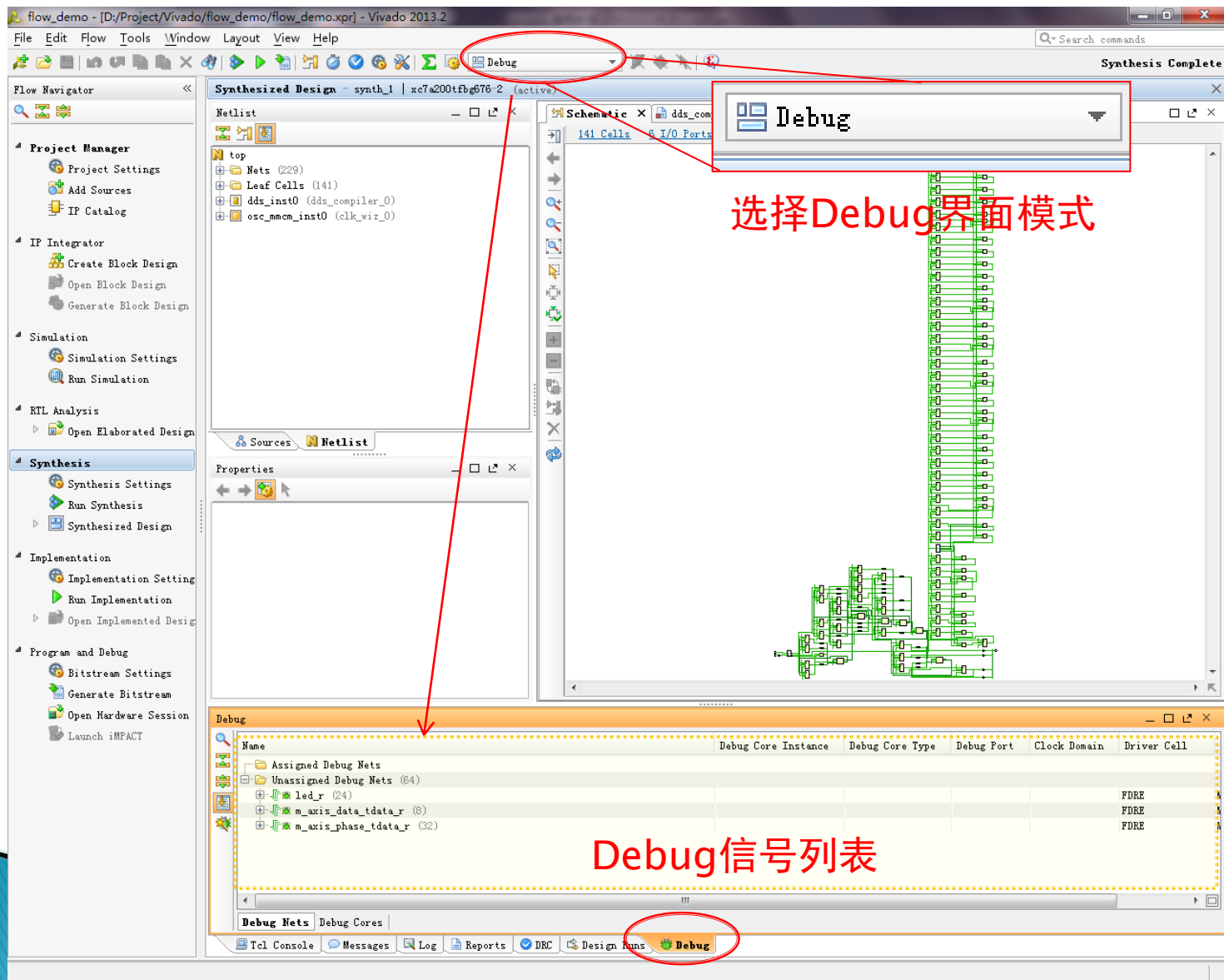
点击它



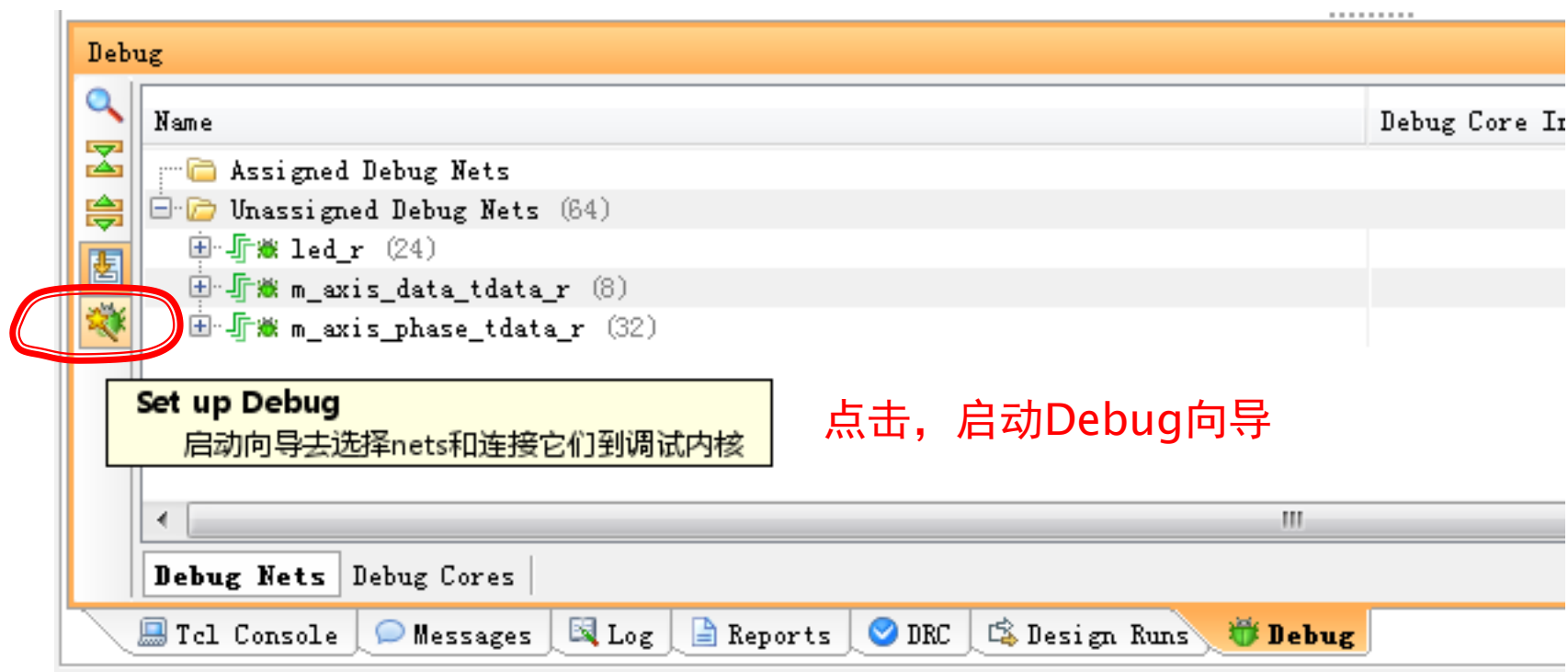
Synthesis完毕



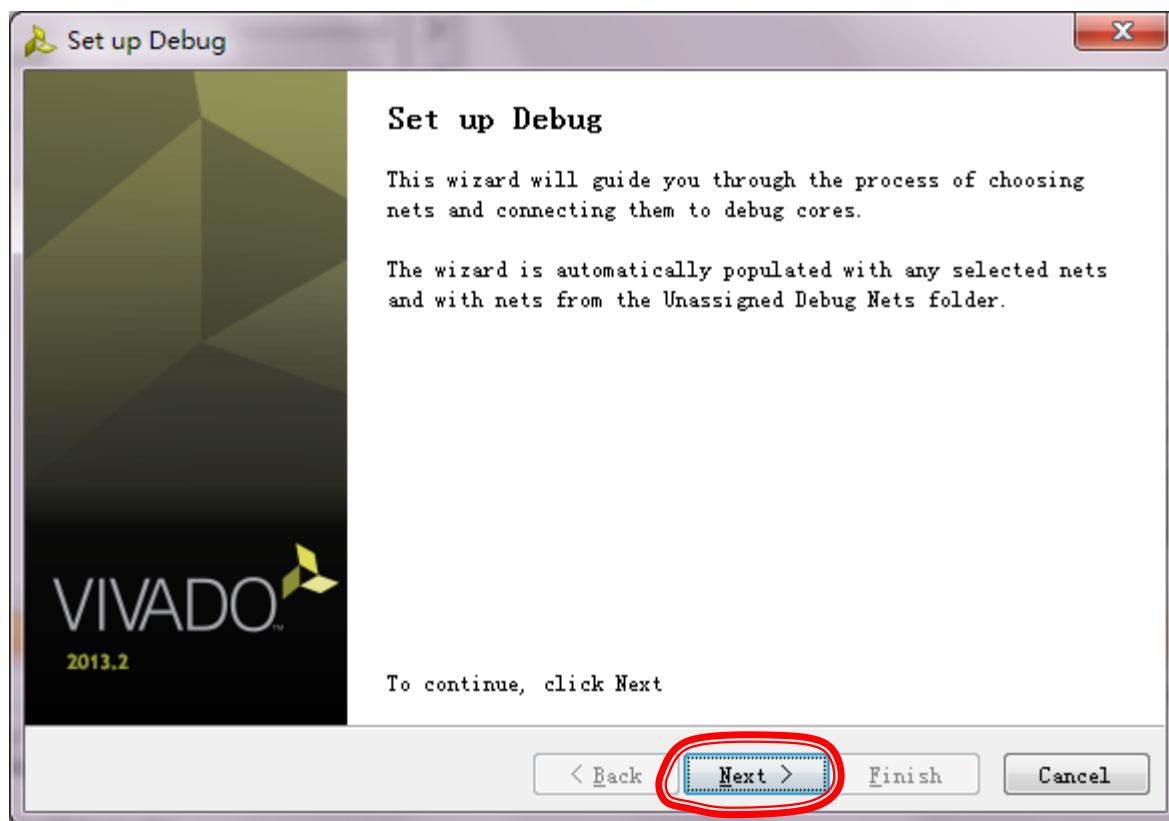
设置Chipscope Debug信号



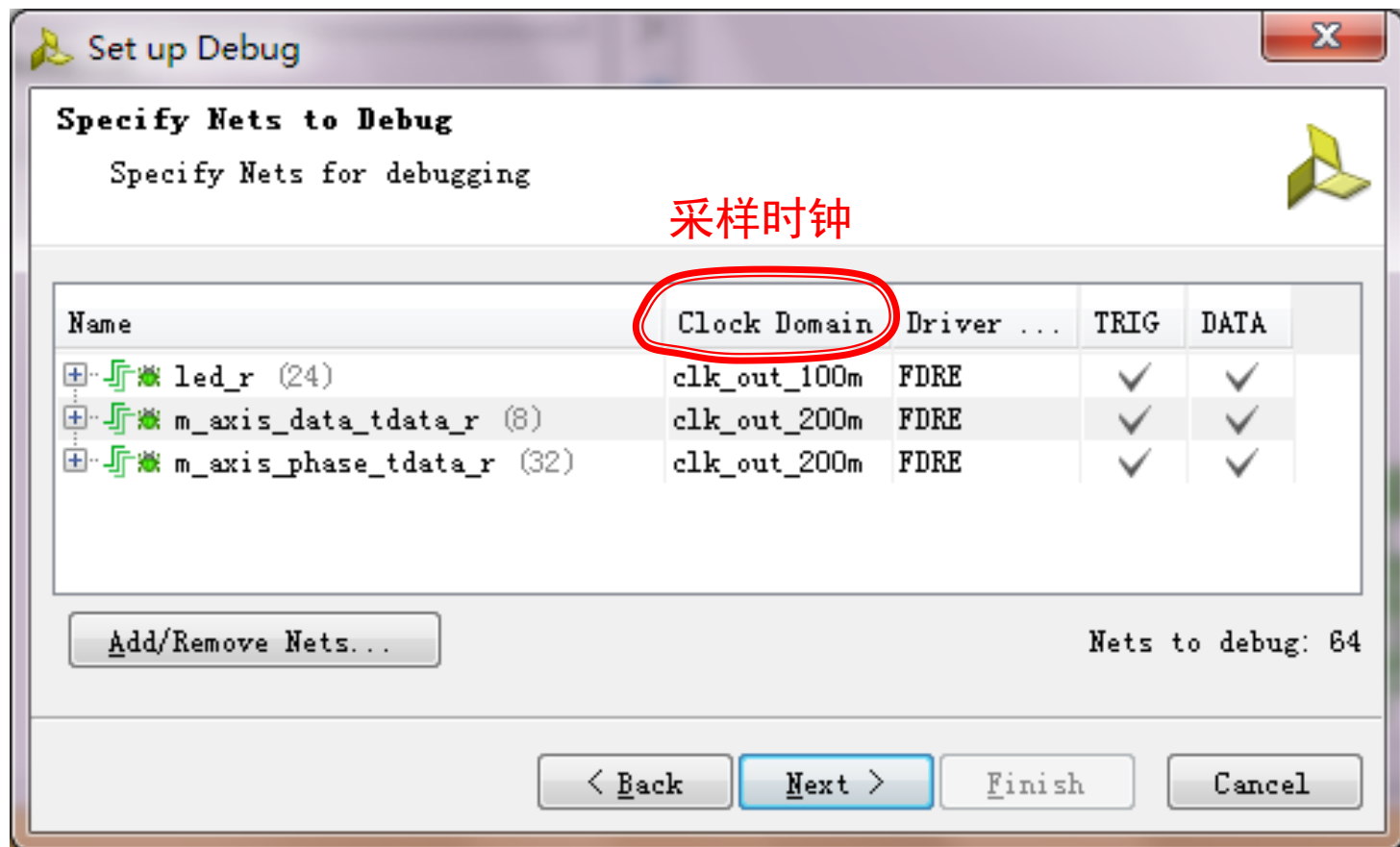
设置Chipscope Debug信号



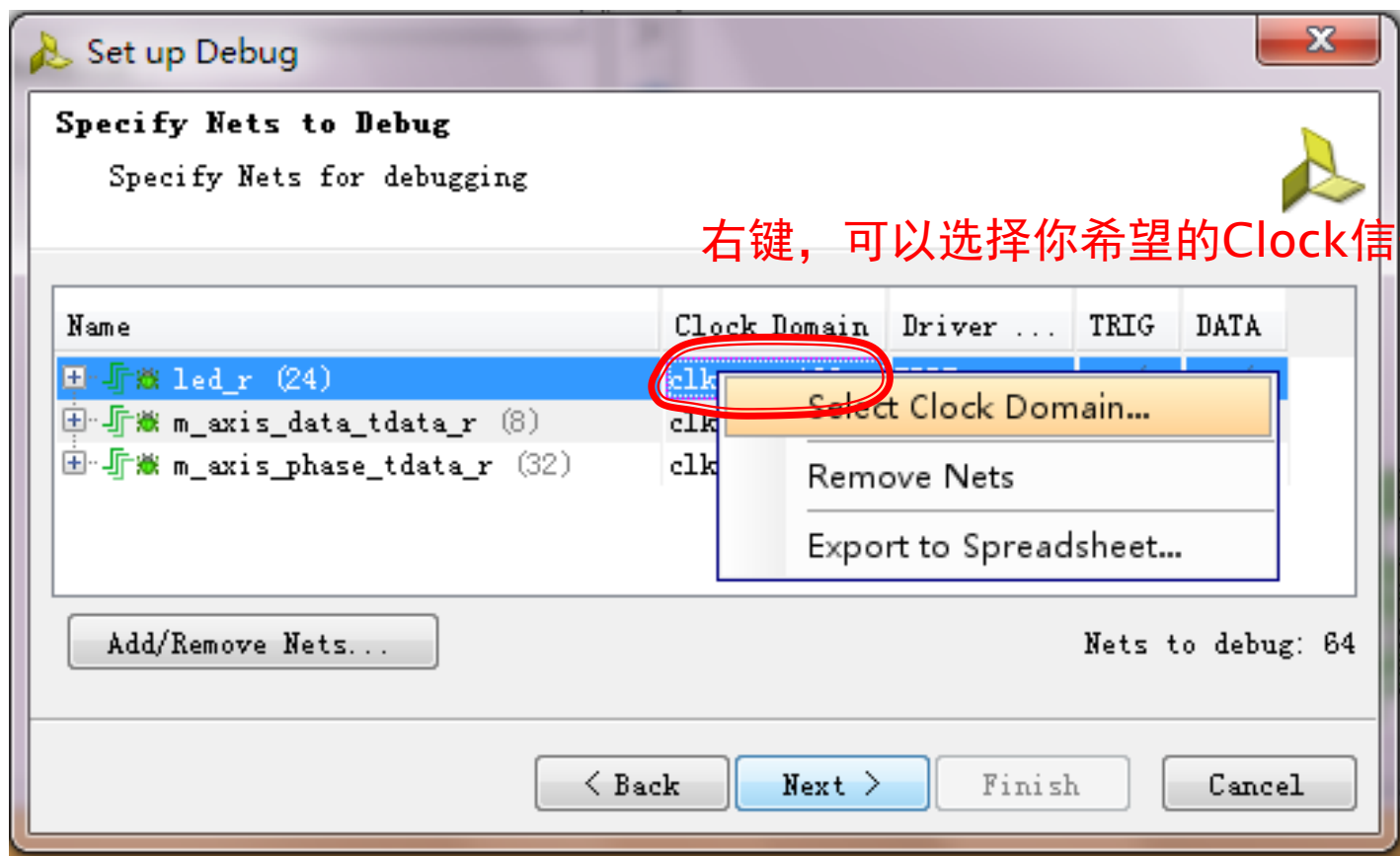
设置Chipscope Debug信号



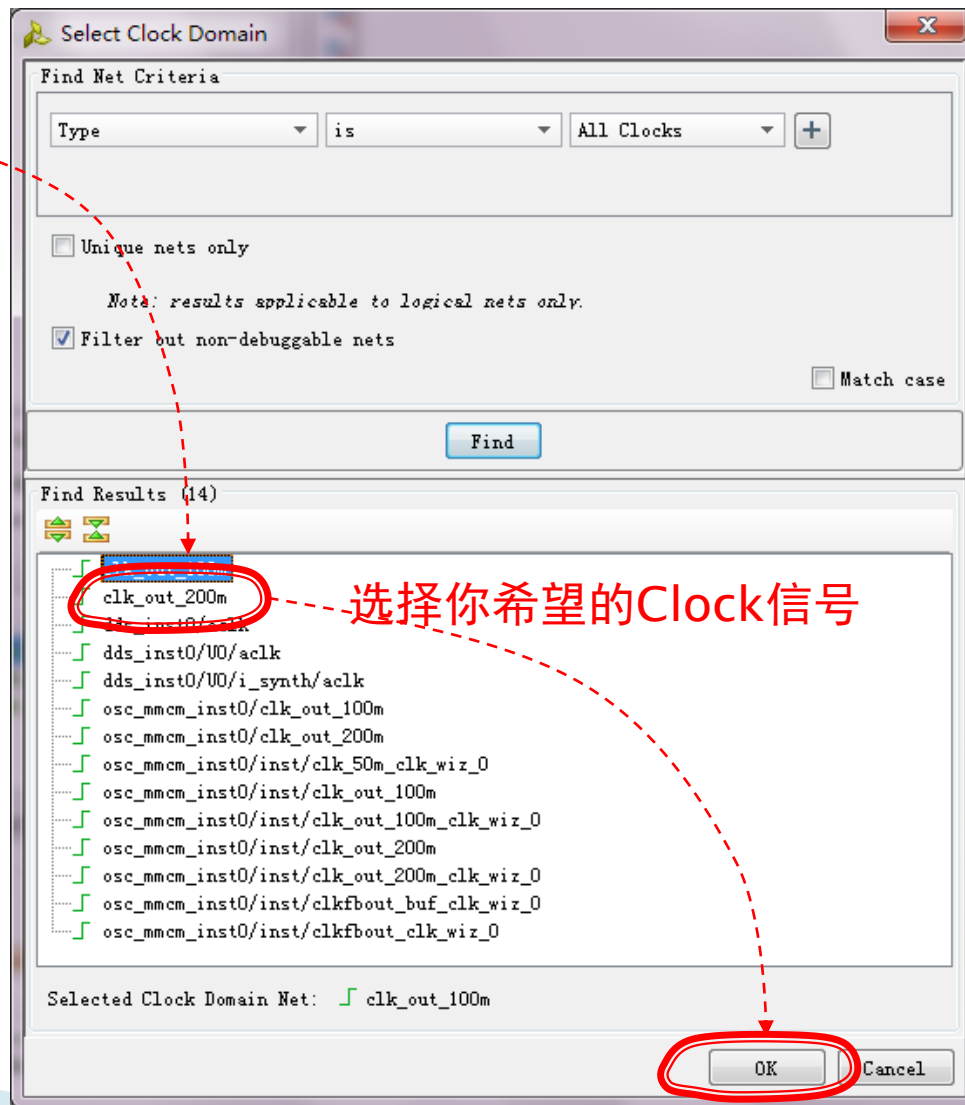
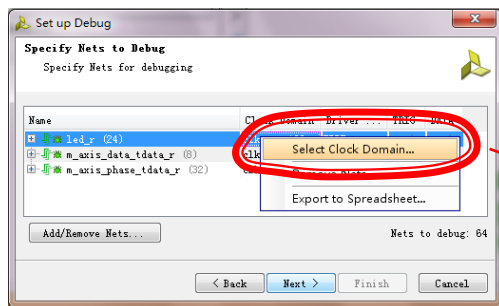
设置Chipscope Debug信号



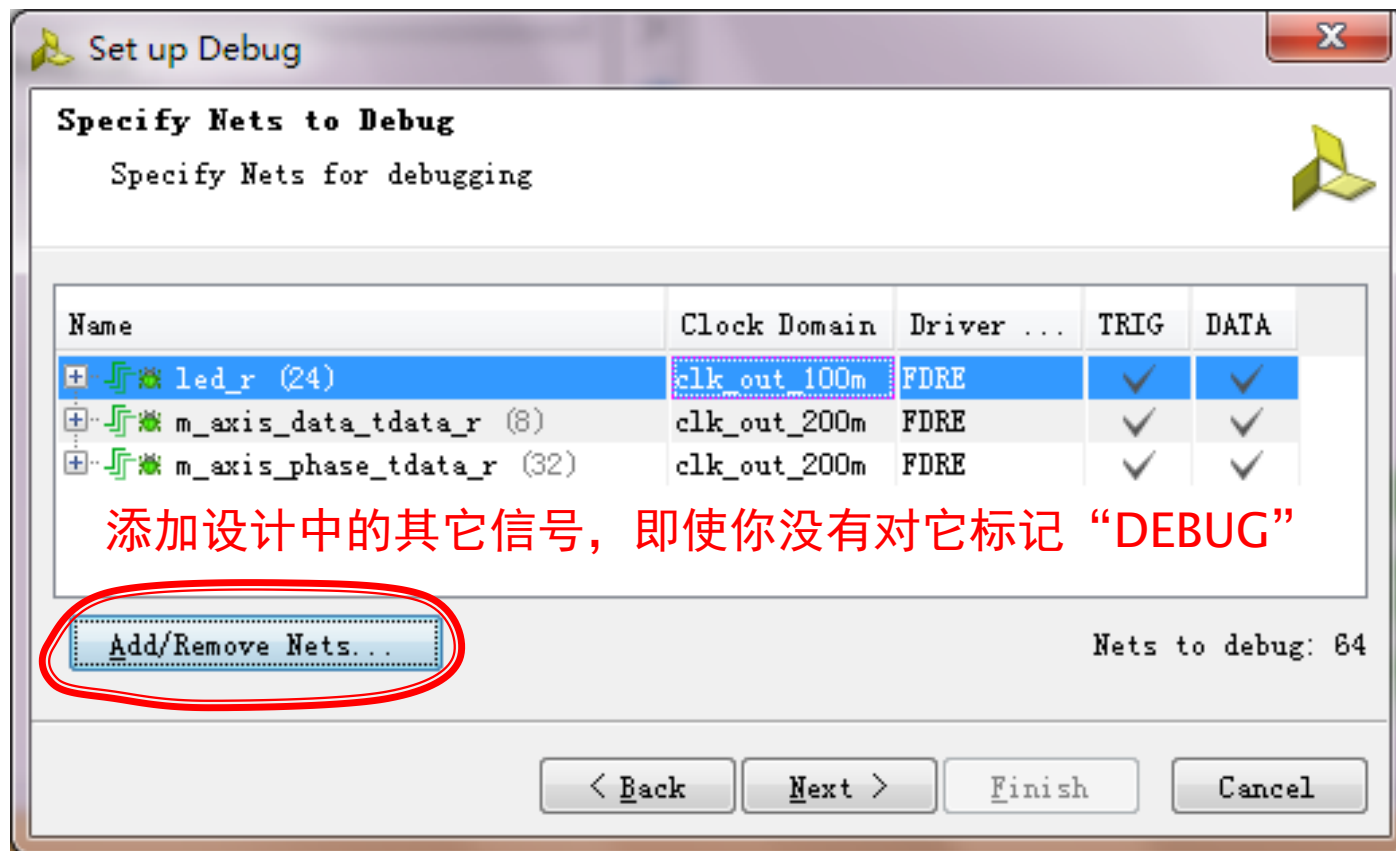
设置Chipscope Debug信号



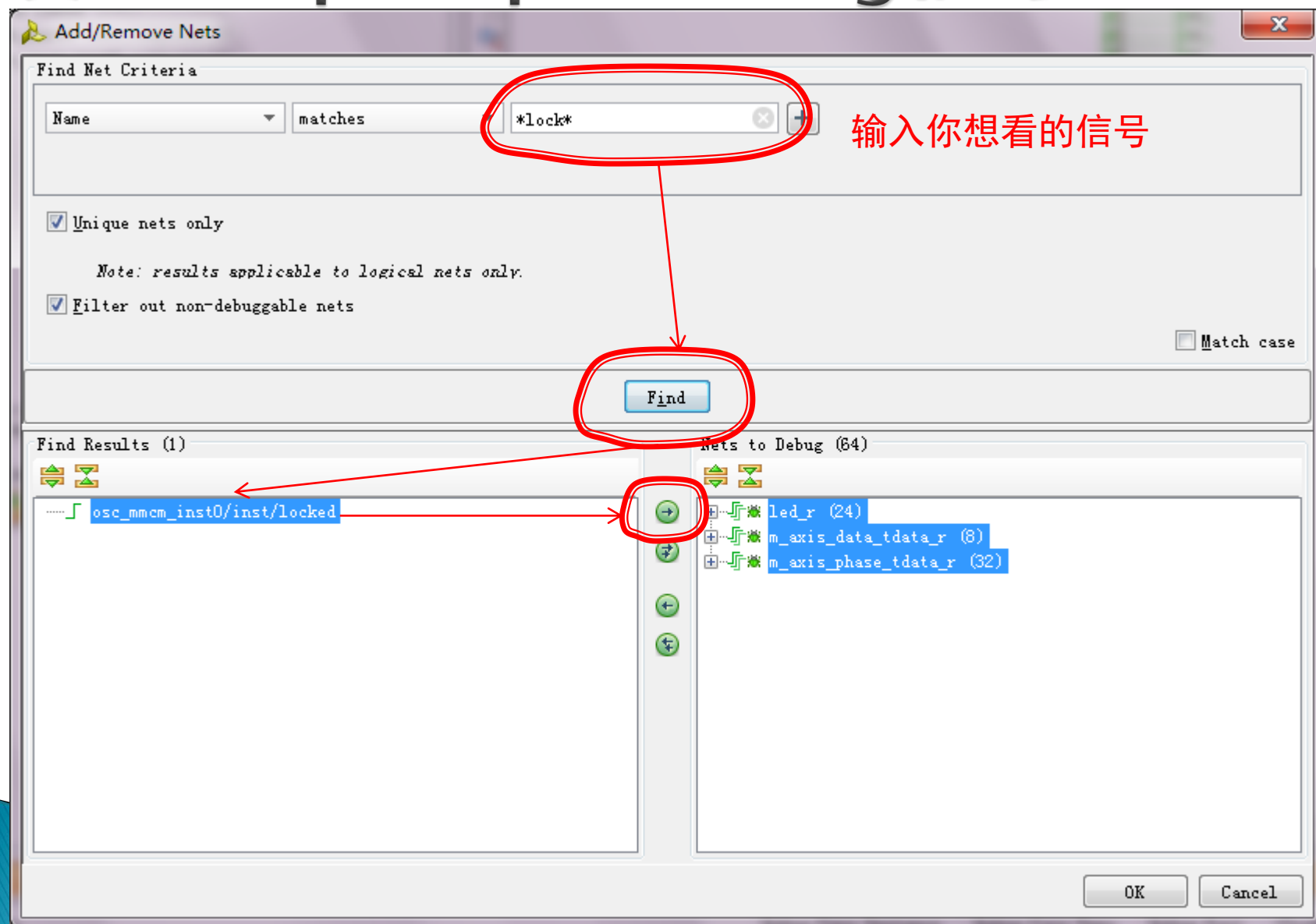
设置Chipscope Debug信号



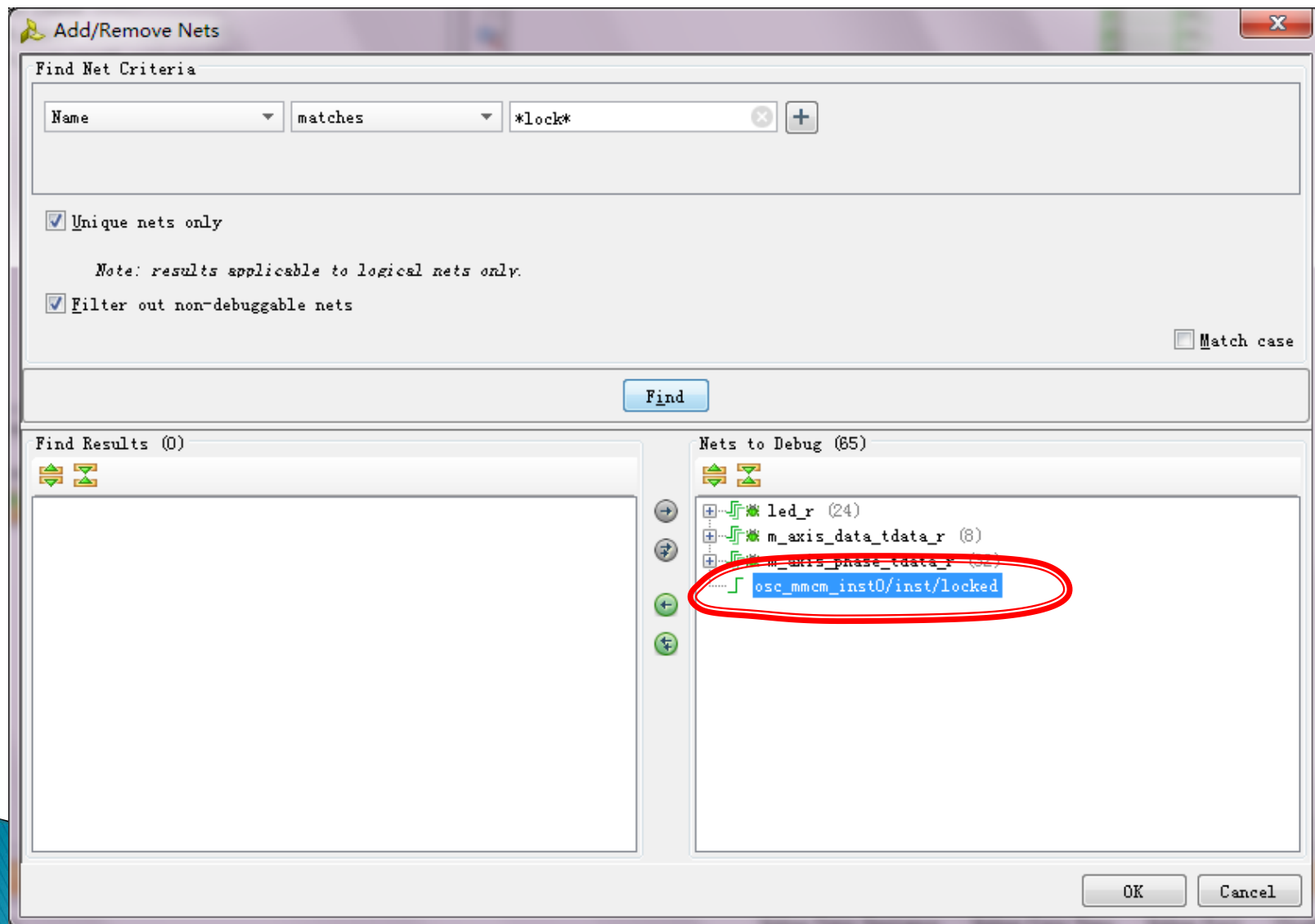
设置Chipscope Debug信号



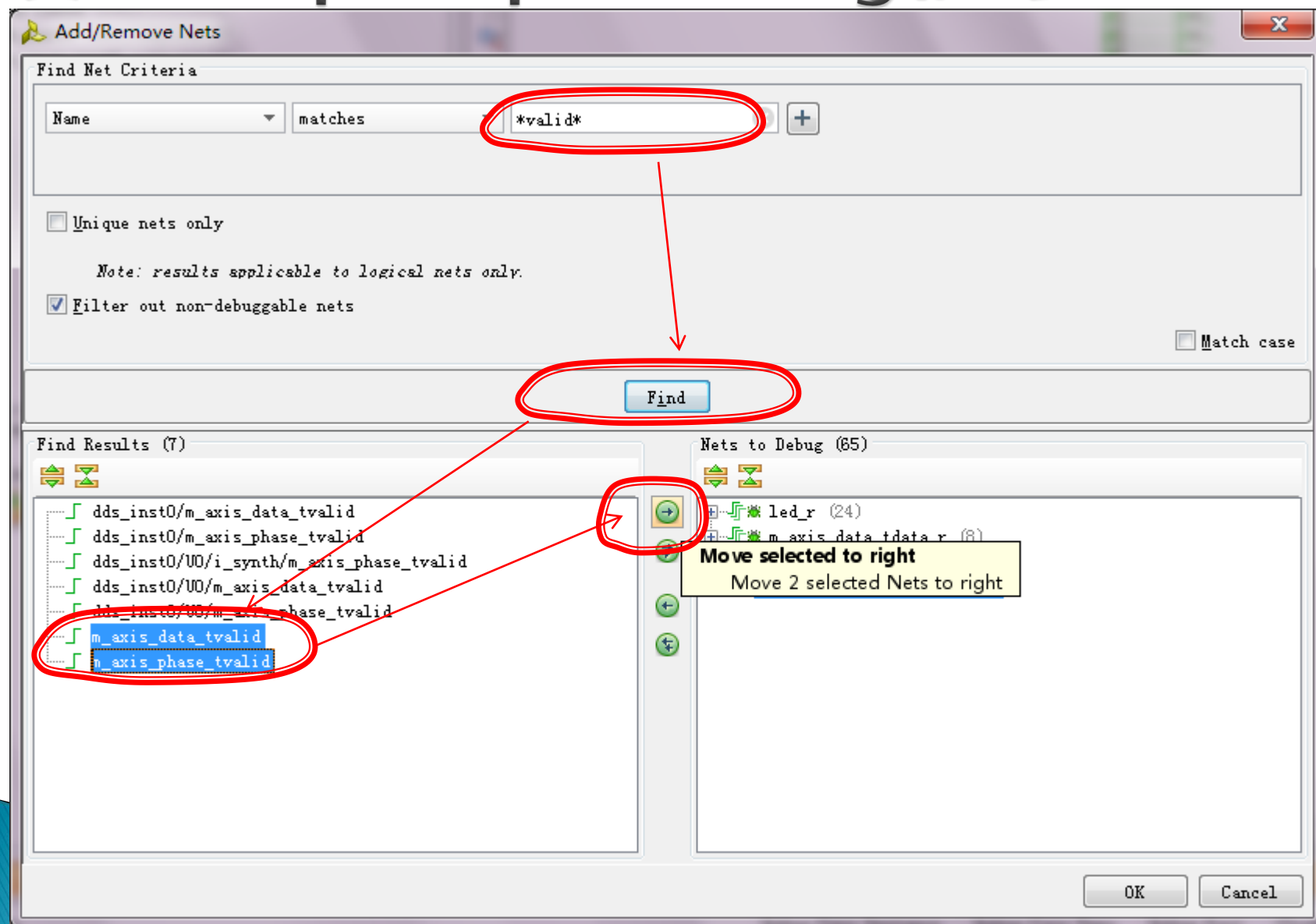
设置Chipscope Debug信号



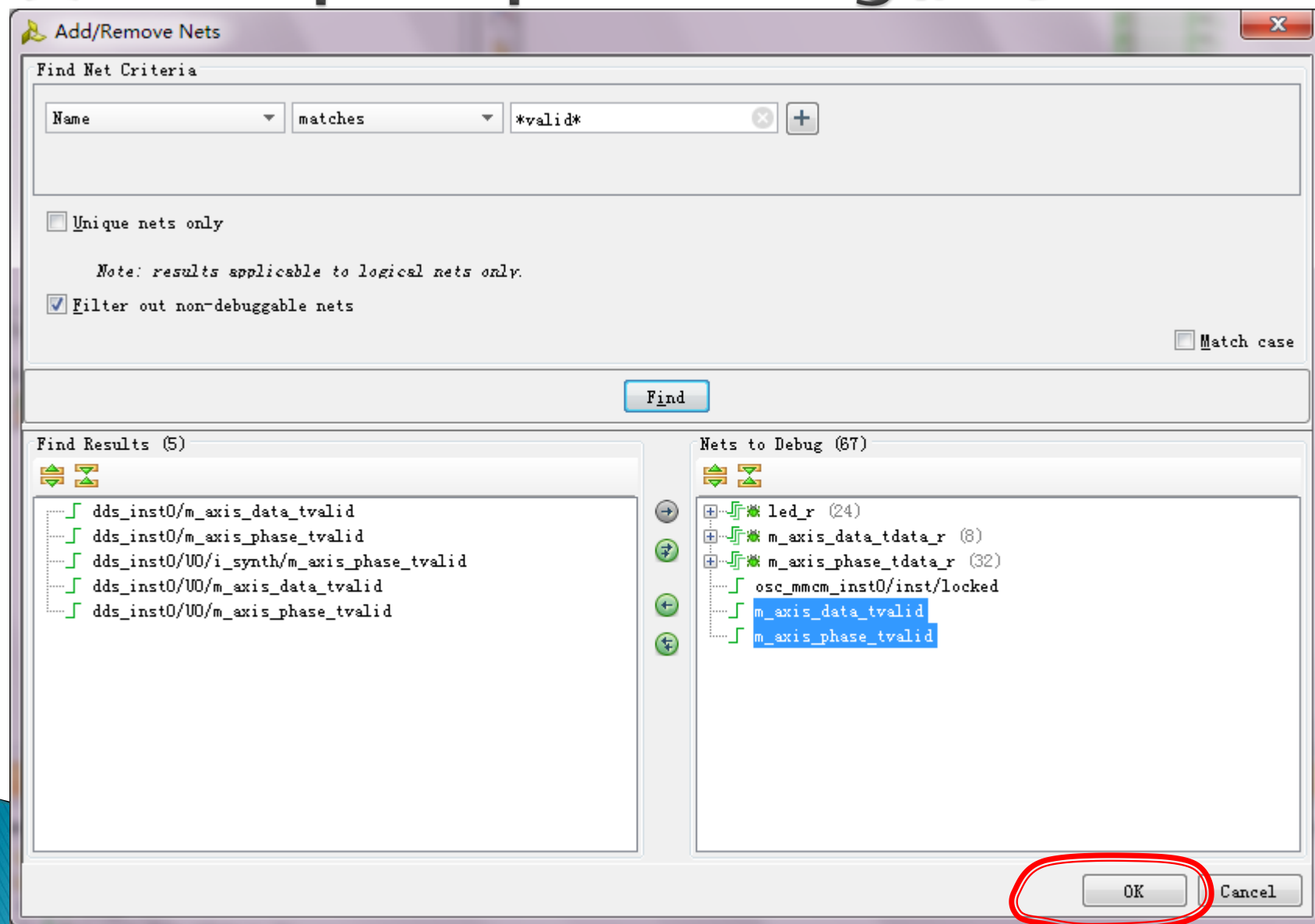
设置Chipscope Debug信号



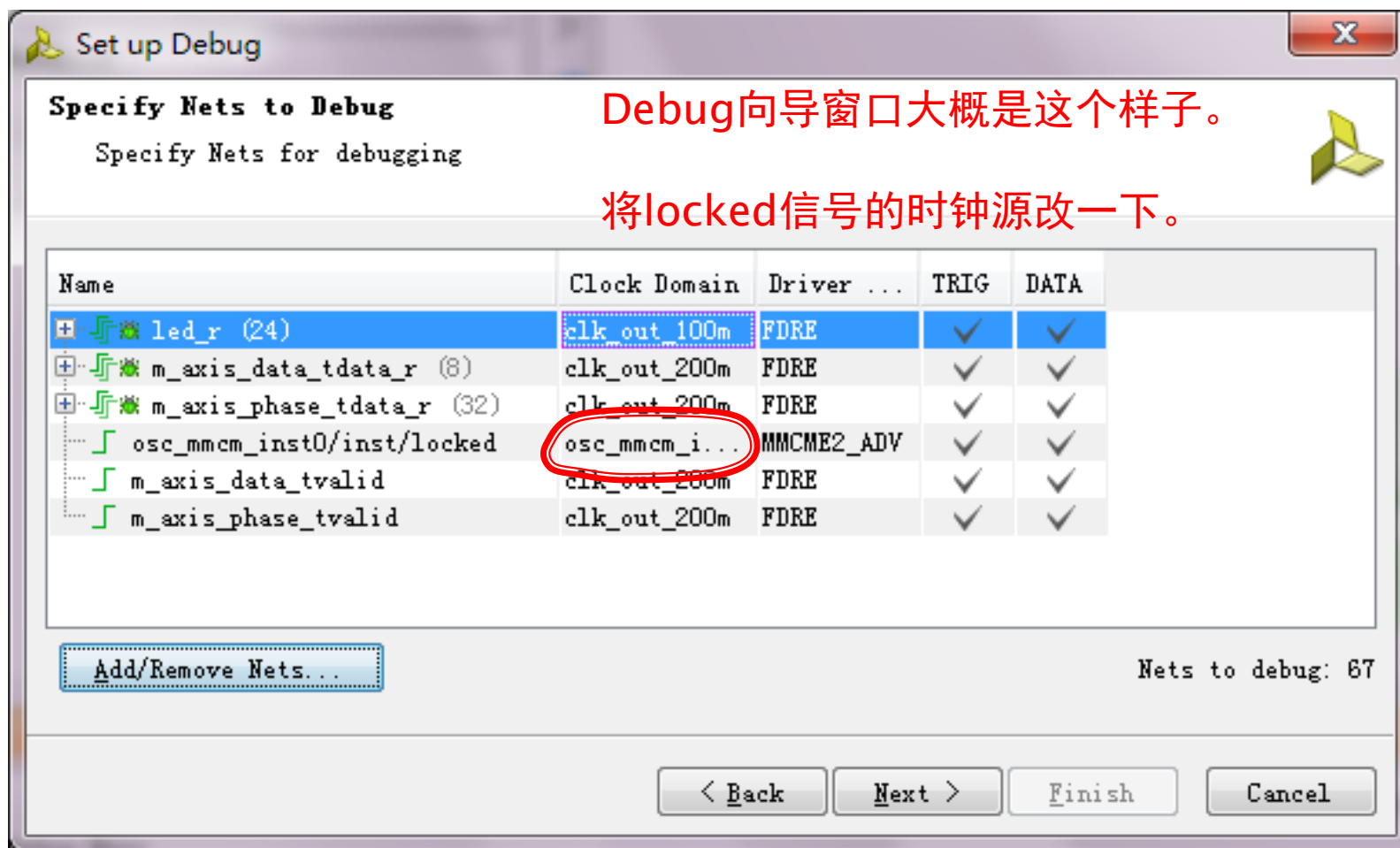
设置Chipscope Debug信号



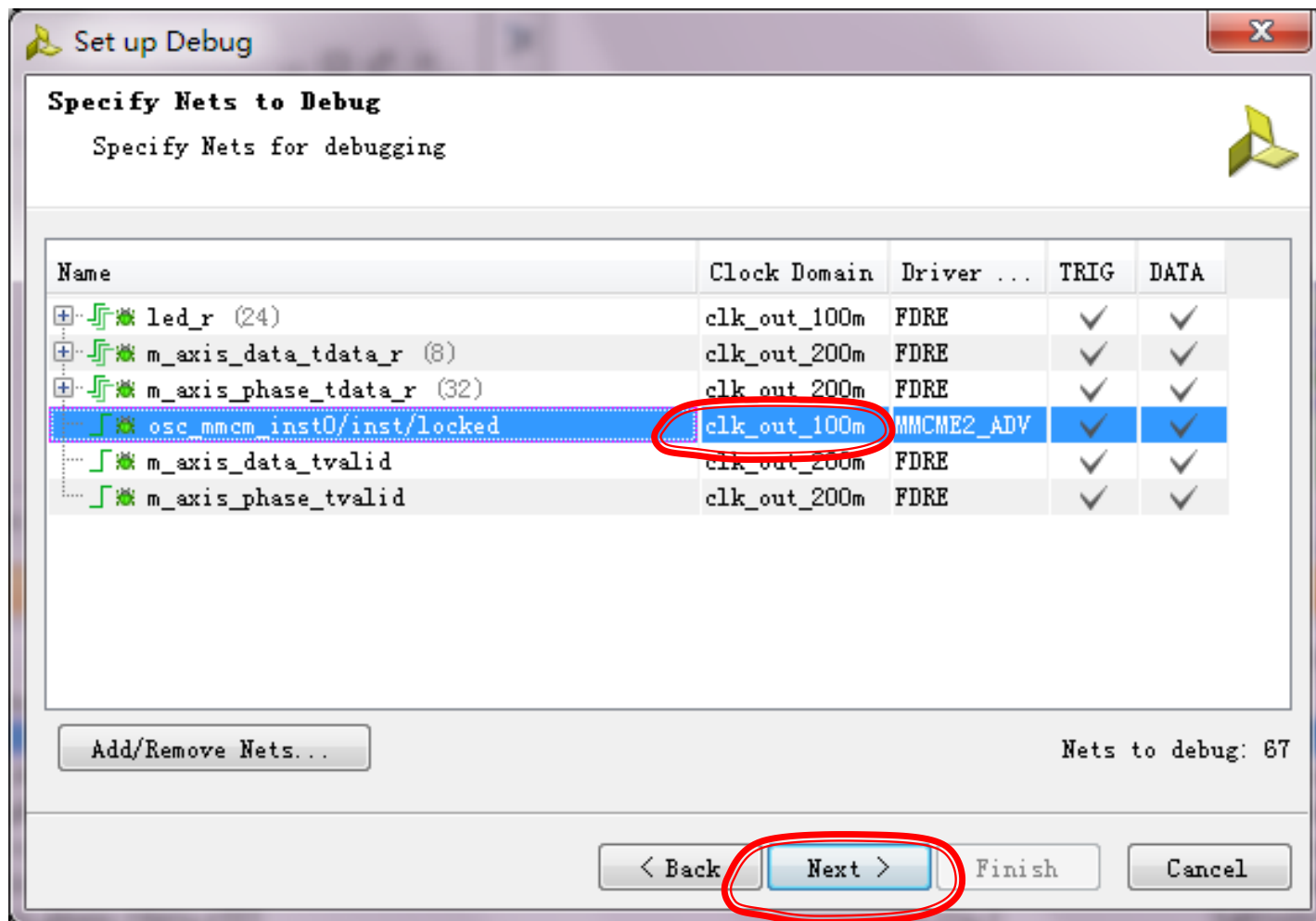
设置Chipscope Debug信号



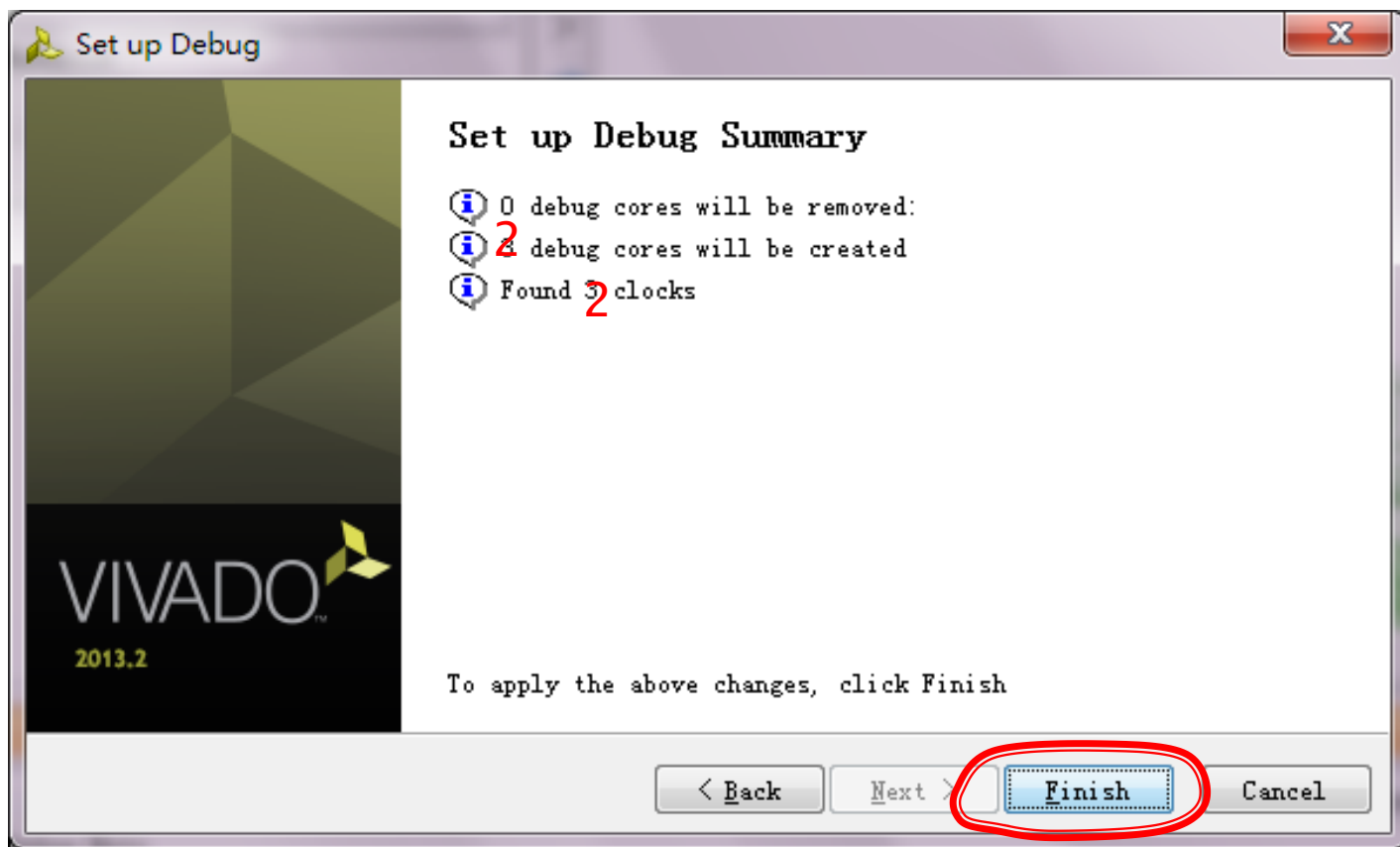
设置Chipscope Debug信号



设置Chipscope Debug信号

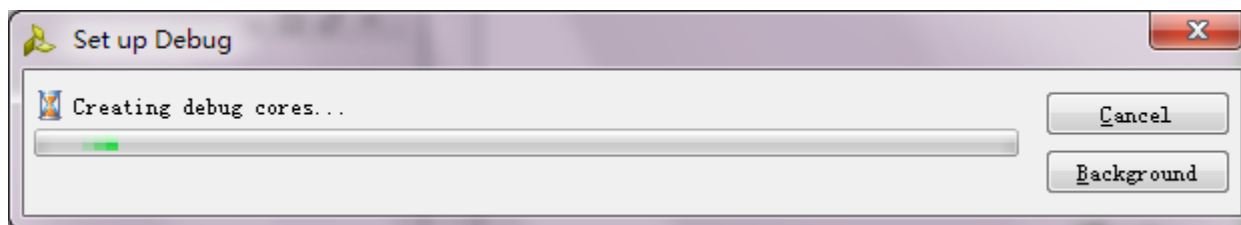


设置Chipscope Debug信号

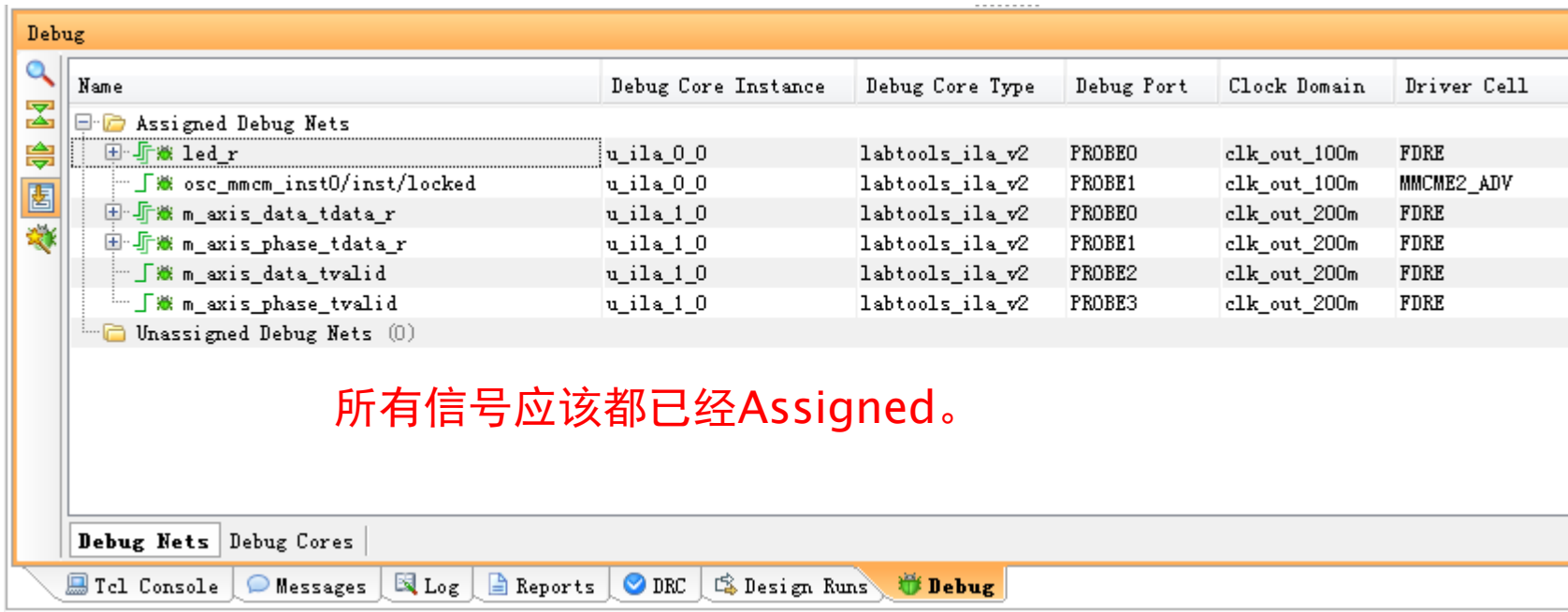


设置Chipscope Debug信号

自动设置Debug相关的core。



设置Chipscope Debug信号

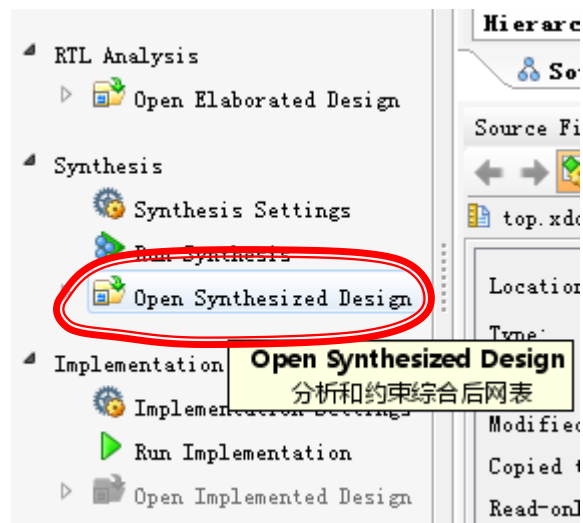


The screenshot shows the 'Debug' window in Xilinx Chipscope. The 'Assigned Debug Nets' section is expanded, showing a list of signals and their corresponding debug core instances, types, ports, clock domains, and driver cells. The signals listed are: led_r, osc_mmc_inst0/inst/locked, m_axis_data_tdata_r, m_axis_phase_tdata_r, m_axis_data_tvalid, and m_axis_phase_tvalid. All signals are assigned to the 'labtools_ila_v2' debug core type. The clock domains are 'clk_out_100m' for the first two signals and 'clk_out_200m' for the remaining four. The driver cells are 'FDRE' for all signals. The 'Unassigned Debug Nets' section is empty.

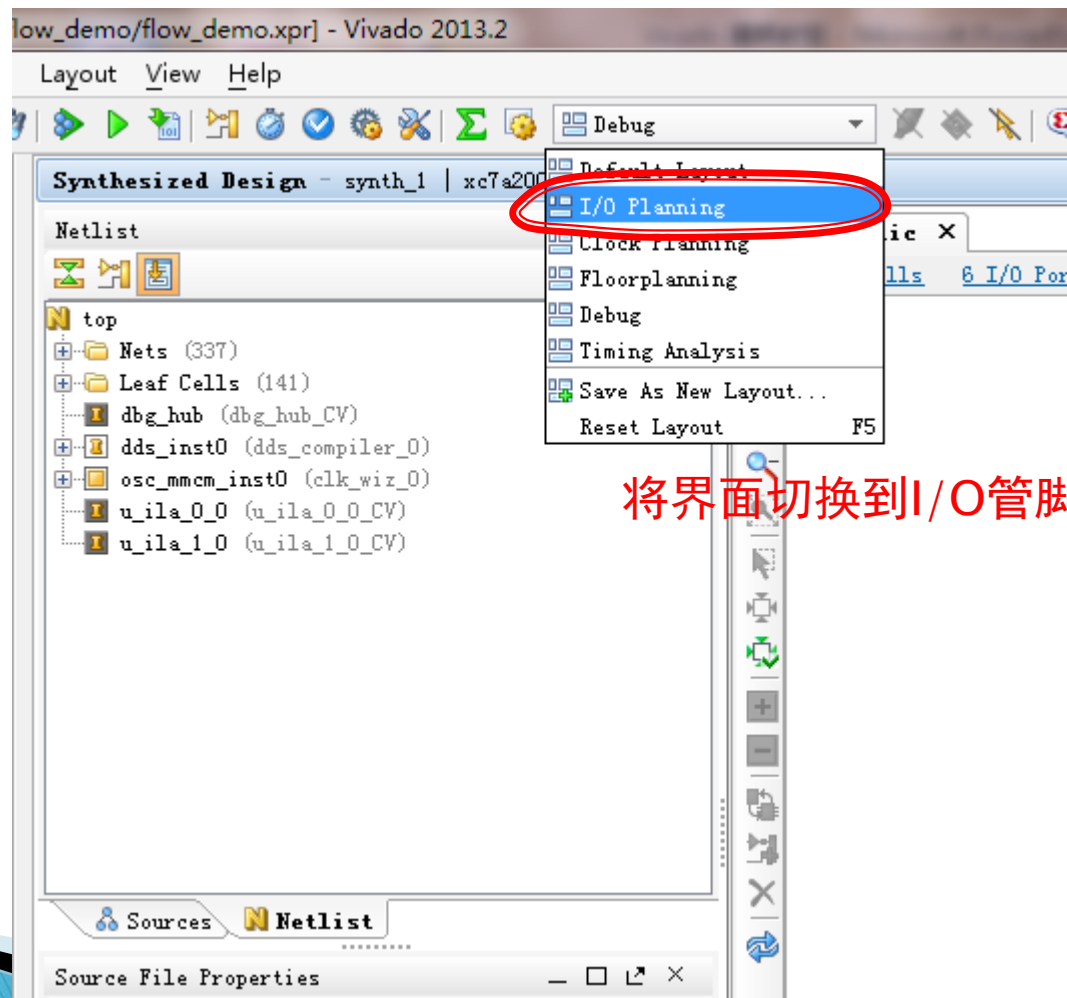
Name	Debug Core Instance	Debug Core Type	Debug Port	Clock Domain	Driver Cell
led_r	u_ila_0_0	labtools_ila_v2	PROBE0	clk_out_100m	FDRE
osc_mmc_inst0/inst/locked	u_ila_0_0	labtools_ila_v2	PROBE1	clk_out_100m	MMCME2_ADV
m_axis_data_tdata_r	u_ila_1_0	labtools_ila_v2	PROBE0	clk_out_200m	FDRE
m_axis_phase_tdata_r	u_ila_1_0	labtools_ila_v2	PROBE1	clk_out_200m	FDRE
m_axis_data_tvalid	u_ila_1_0	labtools_ila_v2	PROBE2	clk_out_200m	FDRE
m_axis_phase_tvalid	u_ila_1_0	labtools_ila_v2	PROBE3	clk_out_200m	FDRE

所有信号应该都已经Assigned。

锁定管脚

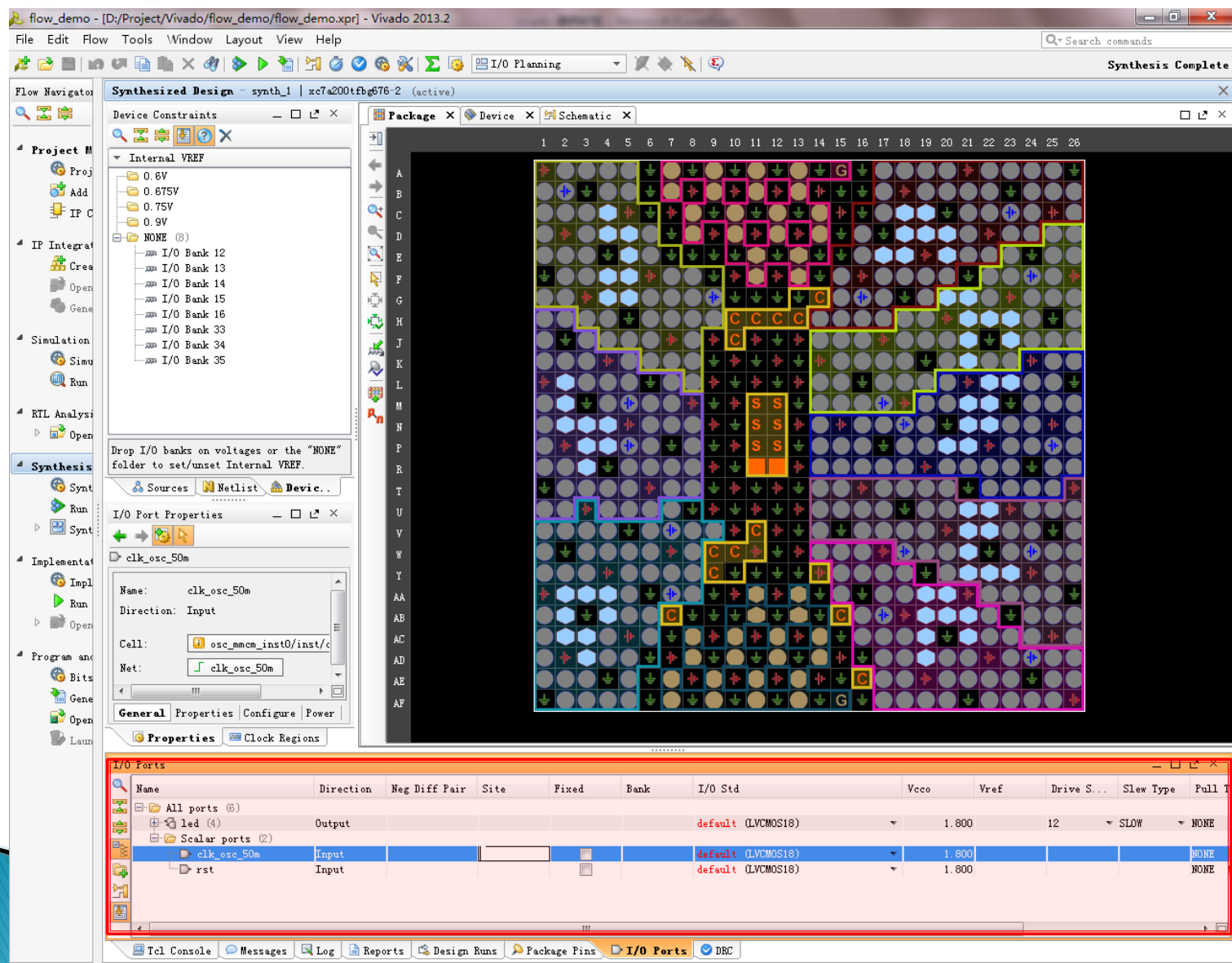


锁定管脚



锁定管脚

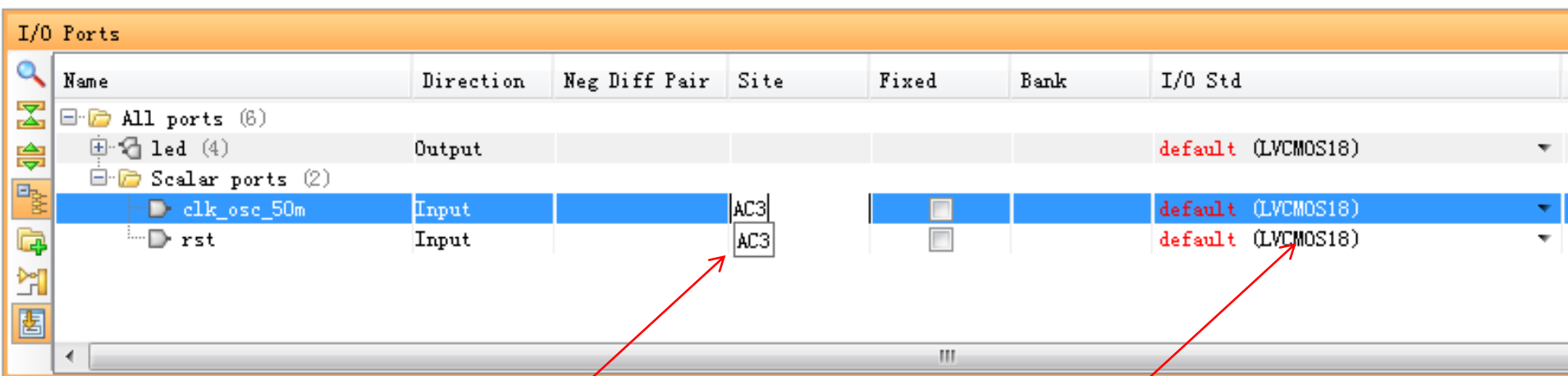
大概是这个样子



主要操作区域

锁定管脚 方法一

- ▶ 直接在界面里面输入管脚位置；
- ▶ 适用于先有硬件，再设计代码；



输入管脚位置
名称，如“AC3”

一定要选择对
应的IO电平

锁定管脚 方法二

- ▶ 将INPUT、OUTPUT信号直接拖放到管脚上；
- ▶ 适用于先有代码，再出原理图、PCB；

锁定管脚 方法二

Top I/O banks on voltages or the "NONE" folder to set/unset Internal VREF.

Sources Netlist Device...

I/O Port Properties

clk_osc_50m

Name: clk_osc_50m

Direction: Input

Site: IOB_X1Y2 ☒ Fixed

Site type: IO_L14P_T2_SRCC_33

Package pin: AC3

General Properties Configure Power

Properties Clock Regions

I/O Ports

Name	Direction	Neg Diff Pair	Site	Fixed	Bank	I/O Std
All ports (6)						
led (4)	Output					default (LVCMOS18)
Scalar ports (2)						
clk_osc_50m	Input		AC3	☒	33	default (LVCMOS18)
rst	Input			☐		default (LVCMOS18)

点按某个信号，直接拖放到管脚上。

一定要选择对应的IO电平

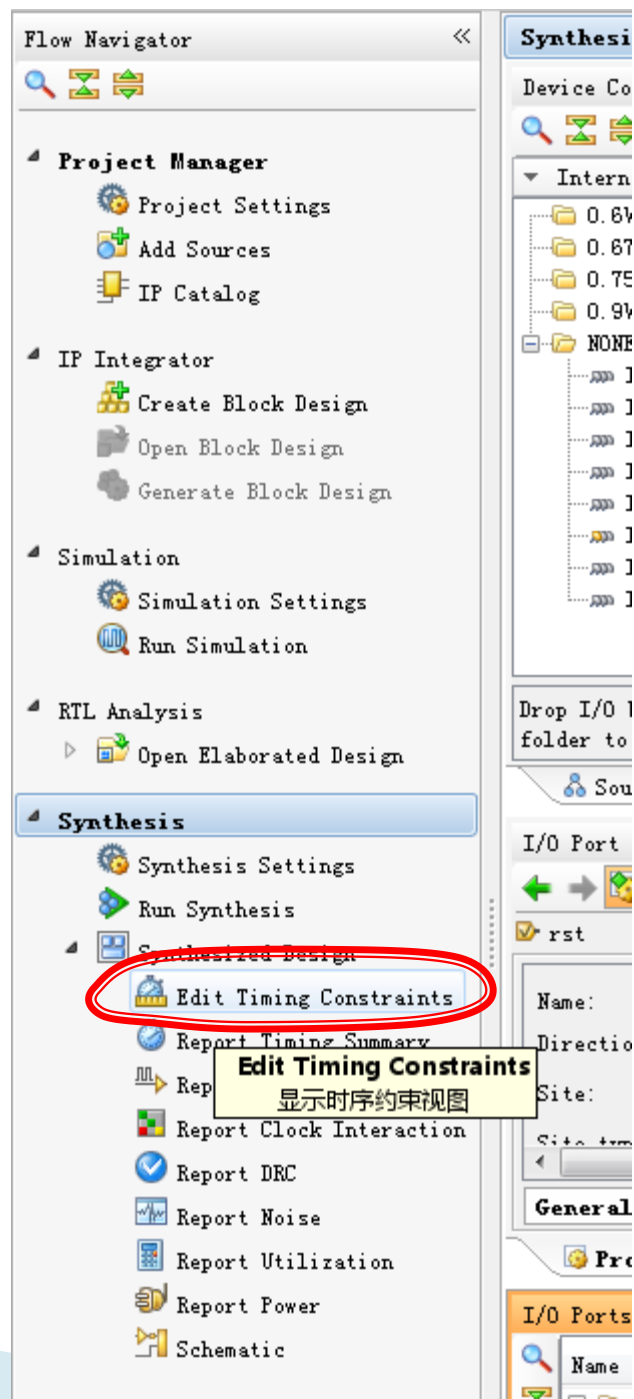
锁定管脚

- ▶ 将设计里的所有管脚分配好。

I/O Ports									
Name	Direction	Neg...	Site	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength
All ports (6)									
led (4)	Output				33	LVC MOS25*	2.500	12	
led[3]	Output		AC2	☒	33	LVC MOS25*	2.500	12	
led[2]	Output		AE2	☒	33	LVC MOS25*	2.500	12	
led[1]	Output		AD1	☒	33	LVC MOS25*	2.500	12	
led[0]	Output		AF2	☒	33	LVC MOS25*	2.500	12	
Scalar ports (2)									
clk_osc_50m	Input		AC3	☒	33	LVC MOS25*	2.500		
rst	Input		AD4	☒	33	LVC MOS25*	2.500		

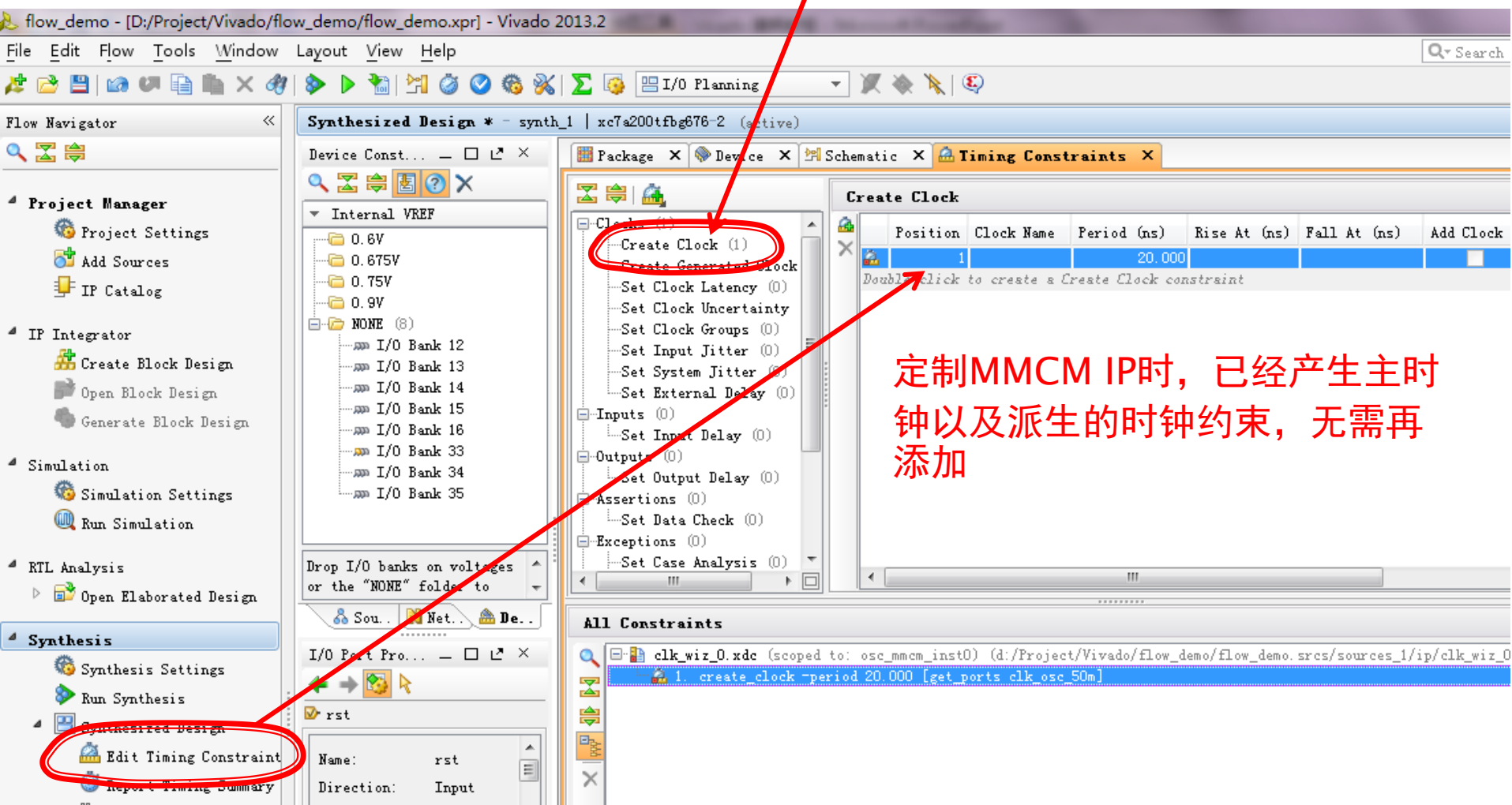
主时钟约束

单击，打开



主时钟约束

如果有其他时钟需要添加，按照 Create Clock向导一步步输入即可。



Save并确认约束

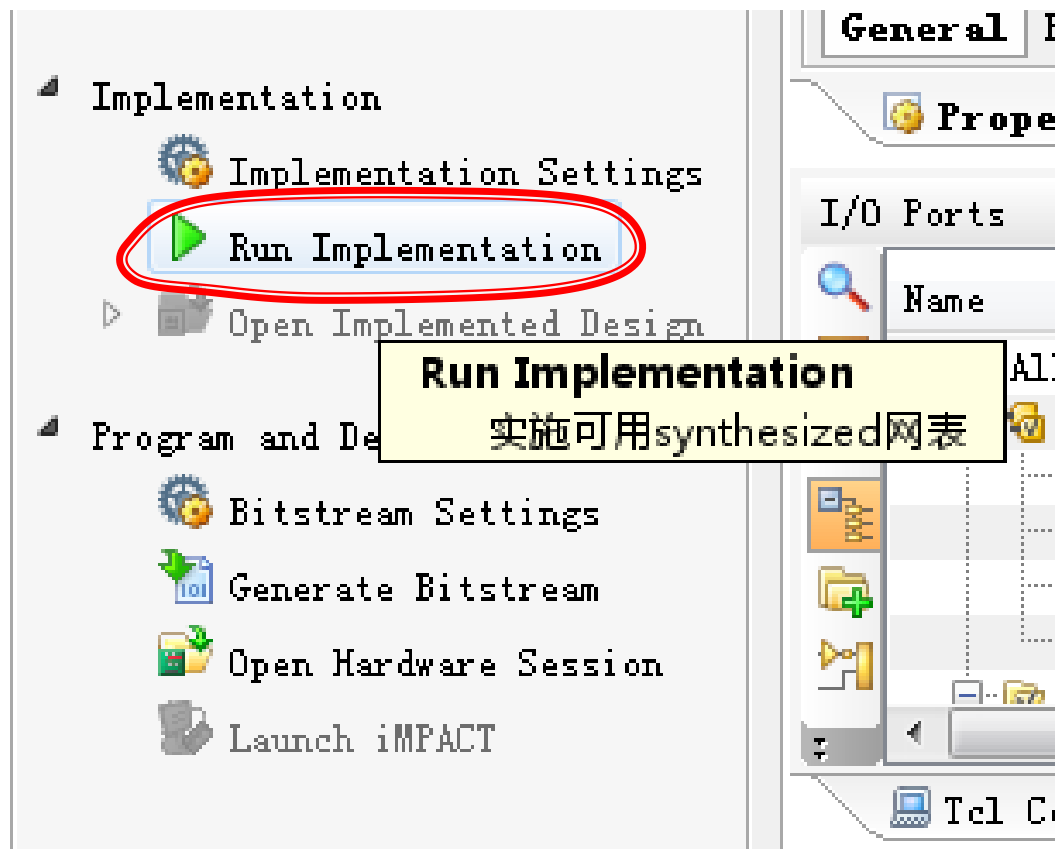
The screenshot shows the Vivado 2013.2 interface. The top toolbar has a red circle around the 'Save' icon (a floppy disk). A red arrow points from this icon to the 'top.xdc (target)' file in the 'Sources' panel under the 'Constraints' folder. The 'Synthesized Design' window is active, showing the 'top.xdc' file with the following content:

```
1 set_property PACKAGE_PIN AC3 [get_ports clk_osc_50m]
2 set_property PACKAGE_PIN AD4 [get_ports rst]
3 set_property PACKAGE_PIN AE2 [get_ports {led[2]}]
4 set_property PACKAGE_PIN AD1 [get_ports {led[1]}]
5 set_property PACKAGE_PIN AF2 [get_ports {led[0]}]
6 set_property PACKAGE_PIN AC2 [get_ports {led[3]}]
7 set_property IOSTANDARD LVCMOS25 [get_ports {led[3]}]
8 set_property IOSTANDARD LVCMOS25 [get_ports {led[2]}]
9 set_property IOSTANDARD LVCMOS25 [get_ports {led[1]}]
10 set_property IOSTANDARD LVCMOS25 [get_ports {led[0]}]
11 set_property IOSTANDARD LVCMOS25 [get_ports clk_osc_50m]
12 set_property IOSTANDARD LVCMOS25 [get_ports rst]
13
```

The bottom status bar shows 'Hierarchy' and 'IP Sources' tabs.

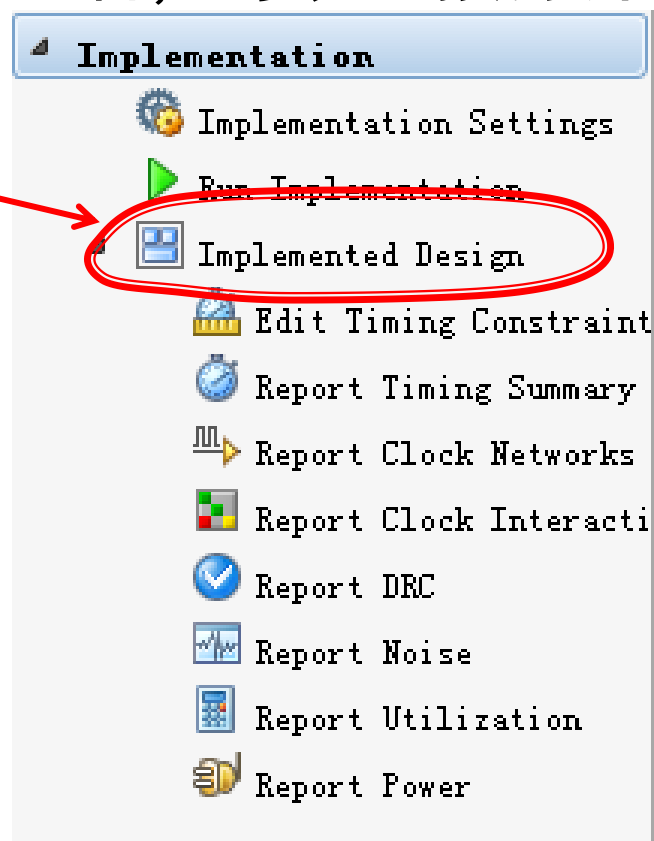
没Save，约束只是存在于内存中，并没有回写到XDC文件中，必须手动Save。

实现设计

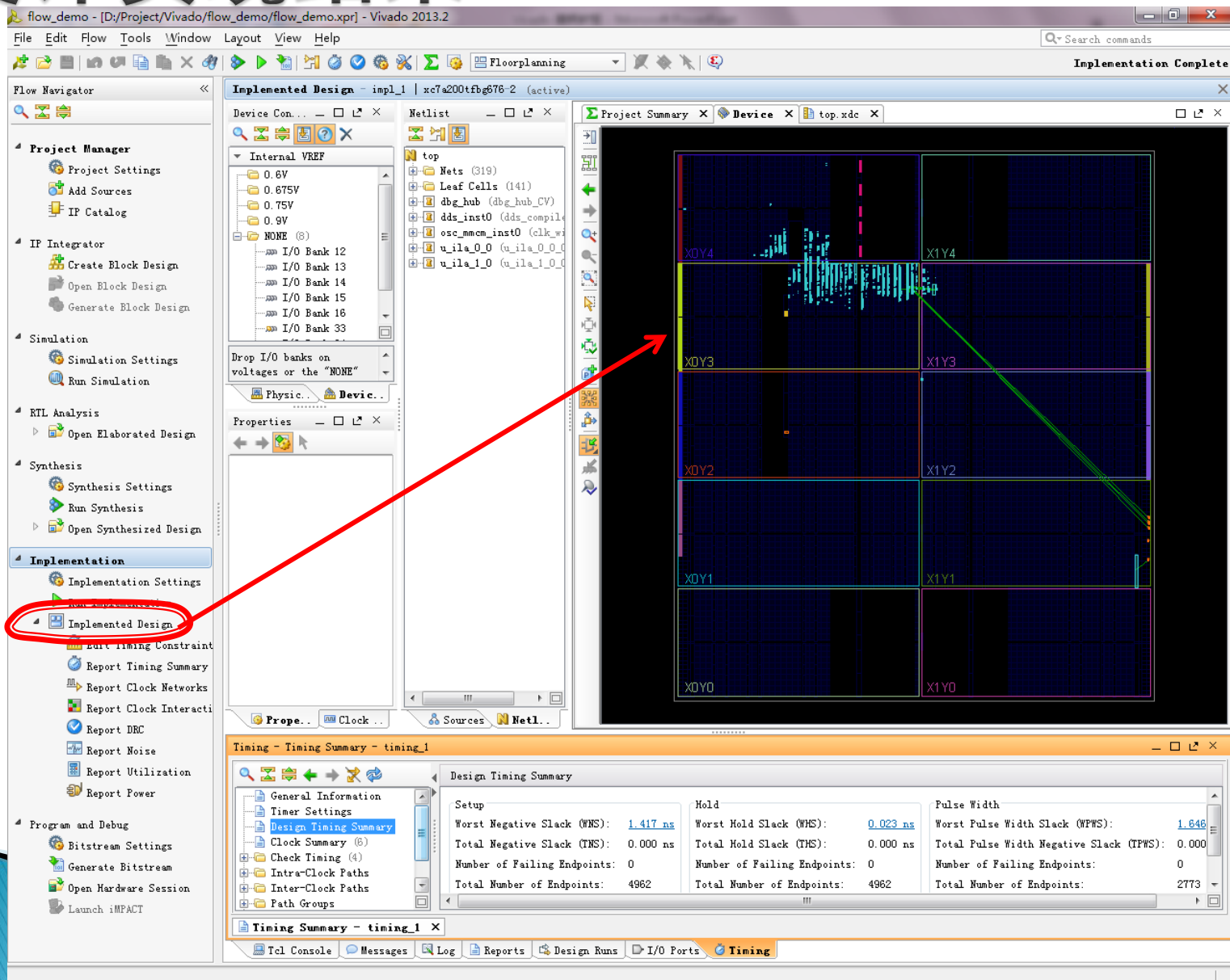


设计实现结果

- ▶ Implement完成之后，可以直接打开Implement结果。
- ▶ 也可以点击

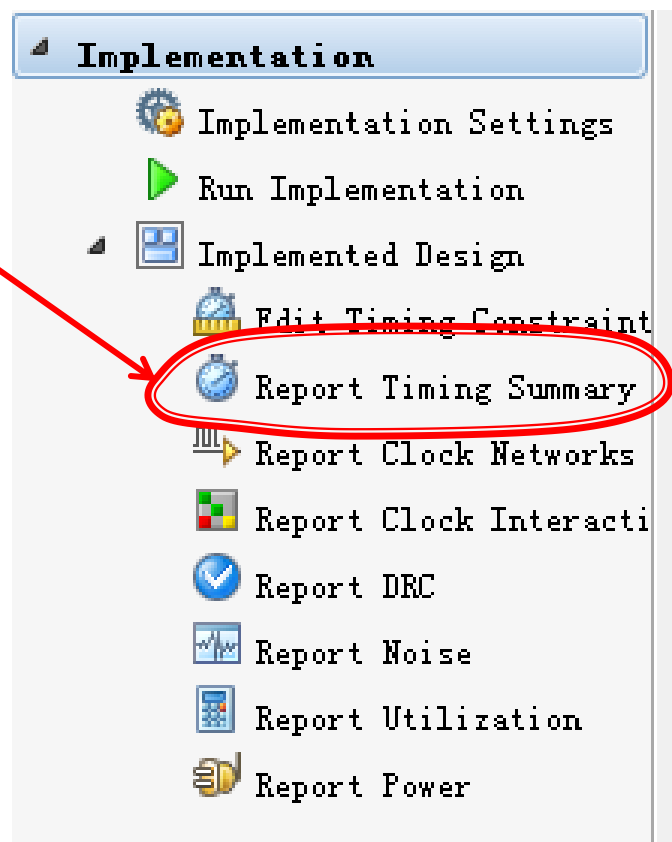


设计实现结果

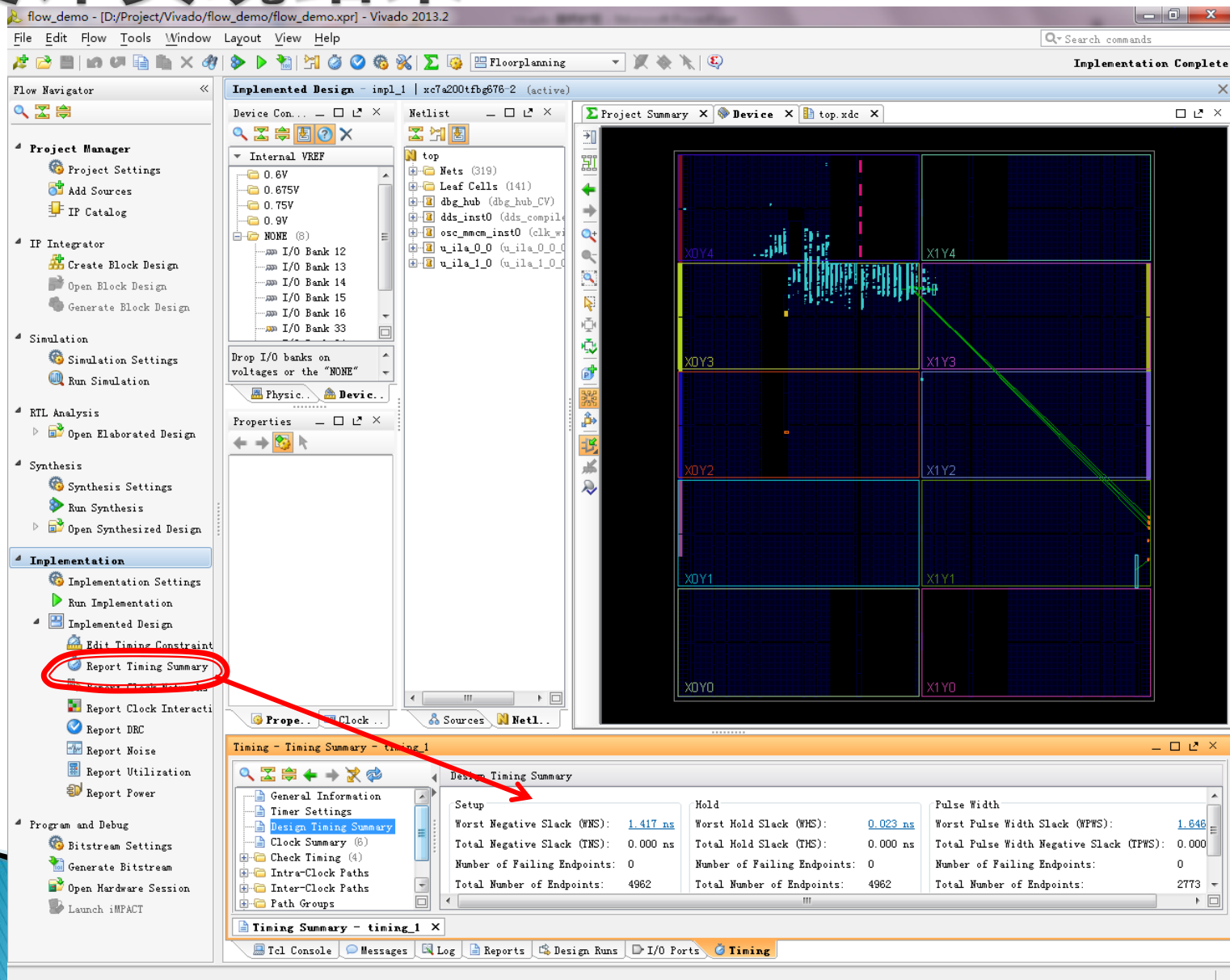


查看时序是否满足

▶ 点击

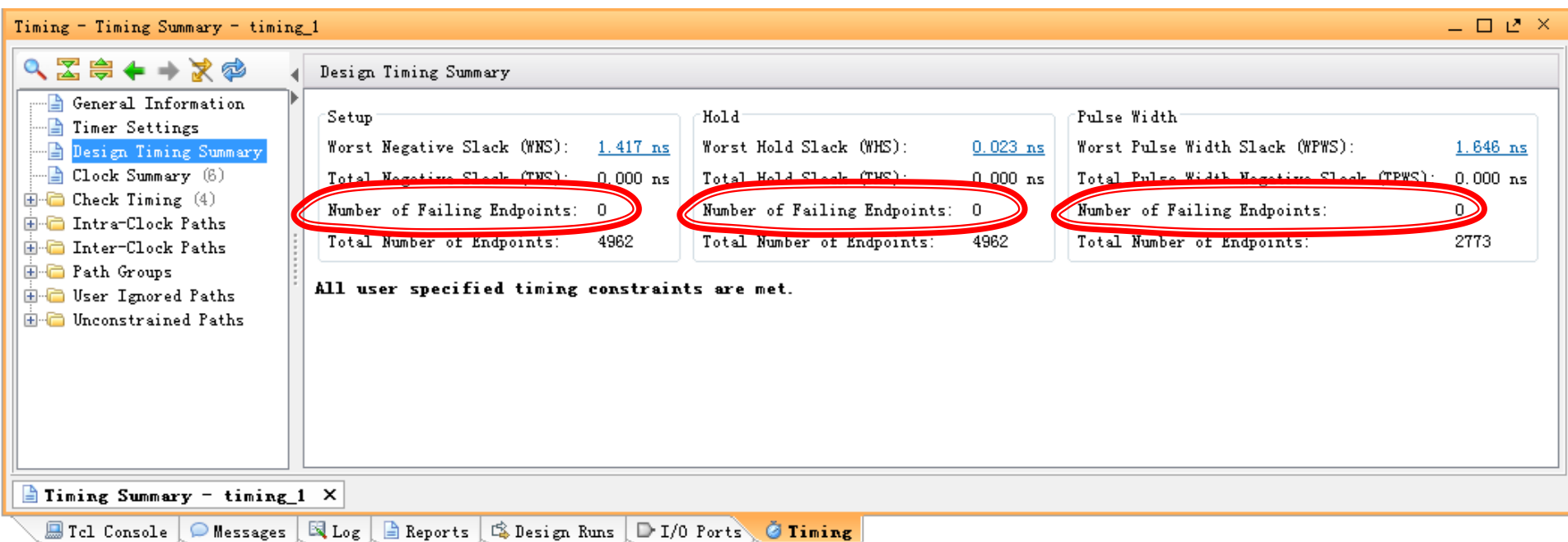


设计实现结果



设计实现结果

- ▶ 不满足的时序会以红色显示



设计实现结果

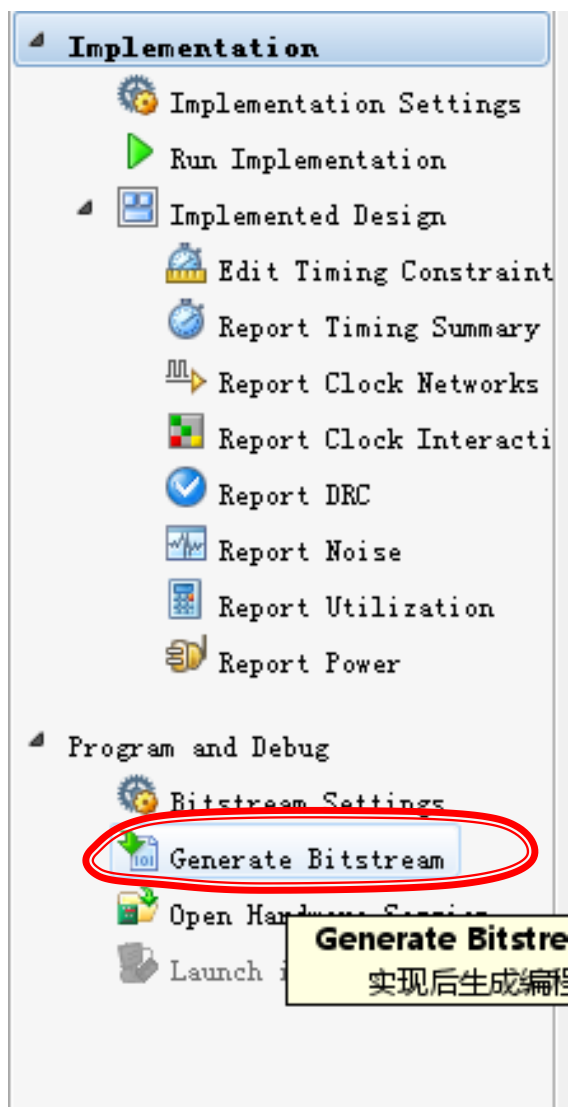
▶ 定制MMCM时钟IP所隐含的时序

Timing - Timing Summary - timing_1

Clock Summary

Name	Waveform	Period (ns)	Frequency (MHz)
clk_osc_50m	{0.000 10.000}	20.000	50.000
clk_out_100m_clk_wiz_0_1	{0.000 5.000}	10.000	100.000
clk_out_200m_clk_wiz_0_1	{0.000 2.500}	5.000	200.000
clkfbout_clk_wiz_0_1	{0.000 10.000}	20.000	50.000
dbg_hub/inst/bscan_inst/SERIES7_BSCAN.bscan_inst/DRCK	{0.000 15.000}	30.000	33.333
dbg_hub/inst/bscan_inst/SERIES7_BSCAN.bscan_inst/UPDATE	{0.000 30.000}	60.000	16.667

生产bit文件



生产bit文件

